

Full length article

CAPRA: Context-Aware patch risk assessment for detecting immature vulnerability in open-source software

Benxiao Tang^a, Shilin Zhang^a, Fei Zhu^a, Aoshuang Ye^{a,b,*}^a School of Computer Science and Artificial Intelligence, Wuhan Textile University, Wuhan, 430200, Hubei, China^b Advanced Cryptography and System Security Key Laboratory of Sichuan Province, Sichuan, China

ARTICLE INFO

Dataset link: <https://github.com/TangBX-wtu/CAPRA>

Keywords:

Open-source software
Defect detection
Immature vulnerability
Static analysis
Patch security
Code property graph

ABSTRACT

Software development increasingly relies on open-source contributions, yet these projects face significant security challenges. Large collaborative codebases frequently encounter vulnerabilities due to varying developer skill levels and reviewers' incomplete understanding of code changes' contextual implications. Traditional detection measures typically activate only after code merging, missing opportunities for detecting potential risks (e.g. immature vulnerability). This paper presents CAPRA, a security detection tool analyzing pending patches through static analysis to identify potential memory leak and Use-After-Free vulnerabilities before integration. Our approach employs code property graph, eliminating compilation environment dependencies while efficiently detecting whether code modifications activate latent vulnerabilities. Using our newly constructed dataset targeting risk-triggering scenarios, experimental results demonstrate CAPRA achieves 97.3% accuracy with 98% recall and only 3.5% false positives—confirming its effectiveness for enhancing code review processes through targeted, early vulnerability detection in rapidly iterating collaborative projects.

1. Introduction

Open-source software has become fundamental to modern software development. Synopsys' 2024 report (Synopsys, 2024) reveals that 96% of audited codebases contain open-source components, with 77% of code in audited libraries coming from open-source projects. However, this integration brings security challenges—84% of codebases contain at least one open-source vulnerability, and high-risk vulnerabilities increased by 54% from 2022 to 2023.

The collaborative development model (Alfayez et al., 2017; Rebouças et al., 2017) characteristic of open-source projects significantly contributes to vulnerability risks (Le et al., 2021). This issue extends beyond open-source communities, as many large enterprises operate internal projects using similar collaborative approaches (Nakayama et al., 2021; Alqudah and Razali, 2016).

Contemporary open-source initiatives often utilize two fundamental approaches to mitigate vulnerabilities: comprehensive manual review procedures (The kernel development community, 2024) and the systematic use of security testing tools (Cai and Knight, 2013; Coccinelle contributors, 2019). Despite the fact that established open-source projects have established community protocols for code contributions, there are considerable challenges that persist due to the nature of the collaborative development model:

Firstly, large-scale open-source initiatives encompass contributors who possess a broad range of security awareness and coding abilities. This heterogeneity frequently will introduce 'immature vulnerabilities (Wu and Lu, 2021)' – potential security risks that do not yet meet the criteria of conventional vulnerability – into the codebase.

Secondly, as projects expand in size and intricacy, both contributors and reviewers encounter difficulties in comprehending the full context of the proposed code modifications. This constraint complicates the code review process (Meneely and Williams, 2009) and exacerbates maintenance challenges (Zajdel et al., 2022), which may result in a decline in code quality.

Despite considerable advancements in the area of software vulnerability detection, such as symbolic execution (Cadaru et al., 2008), taint analysis (Newsome and Song, 2005), fuzzing strategies (Klees et al., 2018), and AI-driven approaches (Li et al., 2021; Wang et al., 2023; Li et al., 2018; Zhou et al., 2019; Lin et al., 2019; Duan et al., 2019; Nguyen et al., 2022), current security tools exhibit limitations in effectively mitigating security threats within collaborative development environments. The shortcomings of these methods largely stem from their post-hoc nature: they necessitate complete detection processes performed independently from the development lifecycle, thereby hindering their applicability during the crucial preliminary stages of code development and review.

* Corresponding author at: School of Computer Science and Artificial Intelligence, Wuhan Textile University, Wuhan, 430200, Hubei, China.

E-mail addresses: bxtang@wtu.edu.cn (B. Tang), 2315363114@mail.wtu.edu.cn (S. Zhang), zf@wtu.edu.cn (F. Zhu), asye@wtu.edu.cn (A. Ye).

In this paper, we propose a front-end static analysis methodology that integrates more closely with the development workflow to address these challenges. Our approach focuses on analyzing how pending code changes affect the execution logic of source files to identify their potential to activate latent vulnerabilities. Based on this approach, we develop CAPRA (Context-Aware Patch Risk Assessment),¹ a prototype static analysis tool specifically designed to detect memory leak and Use-After-Free (UAF) vulnerabilities in C/C++ projects.

The core principle of our approach lies in recognizing that vulnerabilities emerge when various factors in disparate code sections converge, and pending code submissions may unintentionally fulfill conditions that activate hidden vulnerabilities. Utilizing a detection method based on the code property graph (CPG) can greatly lower computational expenses and minimize reliance on the compilation environment.

In addition, we developed a dataset derived from the Software Assurance Reference Dataset (SARD), which includes actual project patches from OpenSSH as well as other projects, alongside instances of ‘hypocrite commits’. The experimental outcomes reveal that CAPRA attains an overall accuracy of 97.3%, with a recall rate nearing 98% and a false positive rate of just 3.5%. These results suggest that CAPRA performs exceptionally well in detecting immature vulnerabilities induced by patches.

In summary, we make the following contributions:

- We introduce an innovative method for assessing patch security, which evaluates the effects of modifications on control logic and data flow by conducting a line-by-line analysis. This method assesses whether alterations may potentially activate immature vulnerabilities. CAPRA serves as a valuable complement to existing static detection methods.
- We present CAPRA, a static analysis tool implemented in Python, crafted to identify memory leak and Use-After-Free (UAF) vulnerabilities that are indirectly introduced in patches.
- We developed a validation dataset specifically centered on scenarios that trigger immature vulnerabilities and performed experiments to assess the effectiveness of CAPRA. The results indicate that CAPRA is effective in identifying potential vulnerabilities that are induced by patches.

2. Preliminaries

2.1. Immature vulnerability

The development of software vulnerabilities is a gradual process requiring multiple triggering conditions. When only some of these conditions exist, we refer to the code as containing an ‘immature vulnerability’. These potential security weaknesses exist in a transitional state—not yet exploitable but positioned to become genuine vulnerabilities when all necessary conditions align.

While some immature vulnerabilities stem from poor coding practices and can be detected by conventional security tools, a significant subset presents a more complex challenge. These latent vulnerabilities neither violate coding standards nor function as actual exploitable weaknesses, allowing them to evade traditional vulnerability scanning mechanisms.

This issue is not limited to isolated incidents. Immature vulnerabilities are widespread throughout the software development landscape. Research by Wu and Lu (2021) has identified more than 5000 immature UAF vulnerabilities in the Linux kernel alone.

Fig. 1 demonstrates a real-world example where a seemingly innocuous patch activated an immature vulnerability. The patch added a `put_device()` function call that released the final reference to a device

(label 1), automatically triggering the reference counting mechanism to free the associated bus when the count reached zero. The vulnerability occurred when `mdiobus_register()` failed and returned an error code (label 2), causing `mdiobus_free()` to execute (label 3), which is a function specifically designed to release the bus. This sequence resulted in a UAF vulnerability that remained undetected in the Linux kernel for five years.

The persistence of this vulnerability over an extended period can be attributed to the patch’s seemingly straightforward and rational nature. Moreover, code reviewers probably overlooked essential contextual details regarding the interaction between the patch and the pre-existing code, indicating a notable deficiency in standard review procedures.

2.2. Patch

In this paper, we define a ‘patch’ as a code modification that is still undergoing review and has not yet been merged into the repository. We specifically focus on minor patches—those containing fewer than 30 lines of code (LOC). This emphasis is well-justified, as previous research has demonstrated that over 70% of bug-fixing patches involve modifications of less than 30 LOC (Li and Paxson, 2017; Wu et al., 2020).

Bug-fixing patches can unintentionally activate underlying problems within the current codebase. This paper focuses on analyzing these interactions during the code review process, prior to the permanent integration of changes into the main codebase.

Minor patches present a unique security challenge despite their limited scope. These small code modifications, precisely because of their targeted bug-fixing purpose and apparent simplicity, often receive less rigorous scrutiny during review. This reduced attention leads reviewers to miss potential interactions between changes and surrounding code.

2.3. Code property graph

We analyze source code for security vulnerabilities using the code property graph—a comprehensive representation of program structure and behavior. The CPG integrates three fundamental program analysis structures (Yamaguchi et al., 2014): the abstract syntax tree (AST) for syntactic structure, the control-flow graph (CFG) for execution paths, and the program dependence graph (PDG) for data and control dependencies. Vulnerability analysis using the CPG has become a leading methodology in static analysis because most vulnerability types cannot be adequately modeled using a single code representation.

3. Approach overview

This section provides a comprehensive overview of our detection architecture and explains the fundamental concepts that underlie our proposed approach.

3.1. System architecture

CAPRA is based on the observation that open-source projects contain numerous potential code defects that may develop into actual vulnerabilities under specific conditions. Fig. 2 shows the detection architecture of CAPRA and its integration method with the standard CI/CD pipeline.

Notation. In this paper, we define the pre-patched file as the ‘source file’ (or F_{pre}), and the post-patched file as the ‘target file’ (or F_{post}). Patch modifications are categorized as ‘added lines’ (present only in the target file) and ‘removed lines’ (present only in the source file). This notation is maintained consistently throughout our methodology description and implementation details to ensure clarity.

During the code review stage, reviewers may overlook comprehensive contextual information about patches due to capacity limitations, inadvertently activating immature vulnerabilities when patches are

¹ <https://github.com/TangBX-wtu/CAPRA>

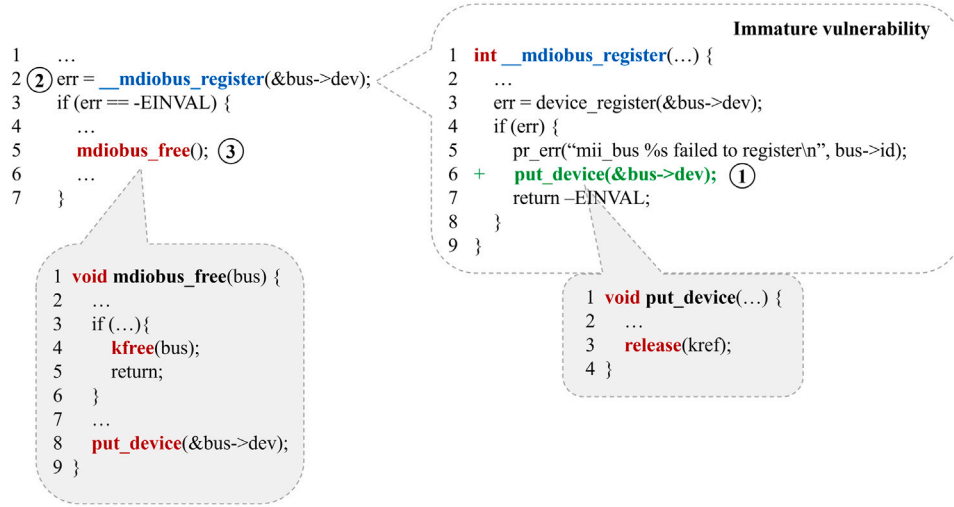


Fig. 1. A real world example (CVE-2019-12819) of UAF vulnerability that was inadvertently triggered by a bug fix patch. The `put_device()` function call added by the patch in `mdibus_register()` releases the `bus` when device registration fails. However, after `mdibus_register()` failure, the program calls `mdibus_free()` which releases the `bus` again.

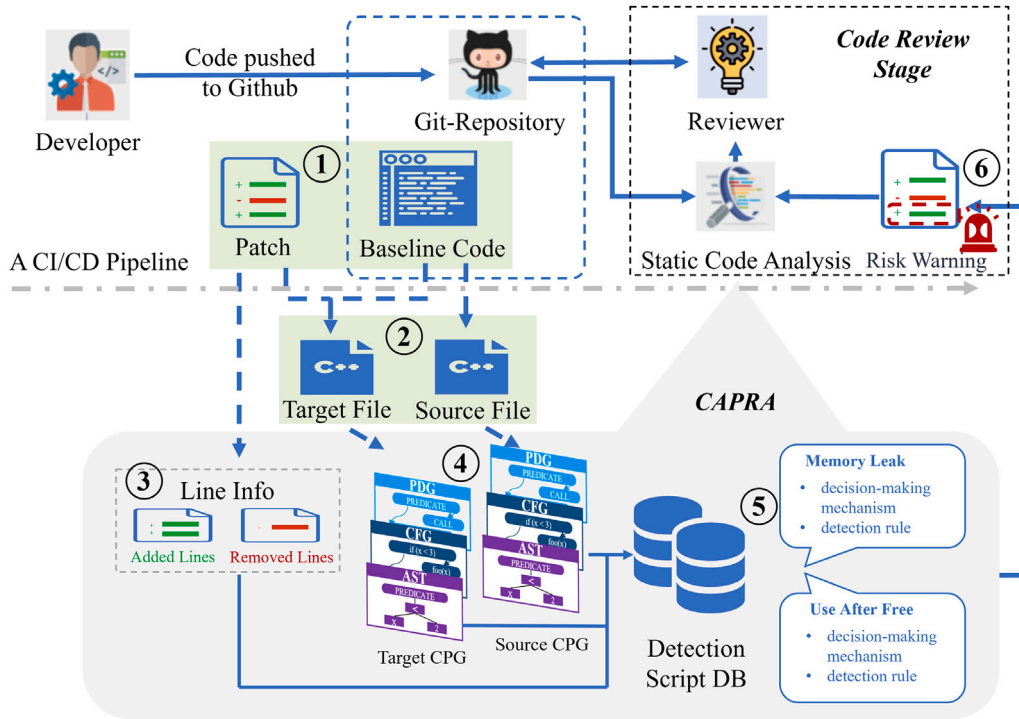


Fig. 2. Overview of CAPRA detection architecture and process. The upper part from left to right shows a standard CI/CD pipeline process, while the lower part displays CAPRA's detection steps. CAPRA is integrated as an extension plugin in the static code analysis section of the code review stage, performing risk detection based on each submitted patch and baseline code.

integrated. Since preventive patches are generally not accepted by development communities (The kernel development community, 2024), CAPRA functions as a plugin deployed during the code review stage. It assists reviewers in identifying potential risks within patches and precisely locating modifications that may introduce defects.

As shown in Fig. 2, CAPRA's detection architecture comprises three main phases: patch analysis (steps 1–3), code property graph generation (step 4), and risk detection (steps 5–6). The primary inputs for CAPRA are the pending patch and the corresponding baseline code that is to be modified (step 1). In step 2, the baseline code and patch are further integrated generate both 'target file' and 'source file'. The detection

process begins by parsing detailed patch information to extract separate lists of added and removed lines (step 3). Next (step 4), CAPRA generates CPGs for both target and source files using Joern (2024). We selected Joern primarily for its robust parsing capabilities and ability to generate CPG without a working build environment and with only partial code availability.

The information about modified lines is then processed line by line by the detection script for risk assessment (step 5). CAPRA ultimately identifies and outputs the specific lines that may introduce potential risks (step 6). This line-by-line detection approach is another key element in maintaining CAPRA's lightweight nature.

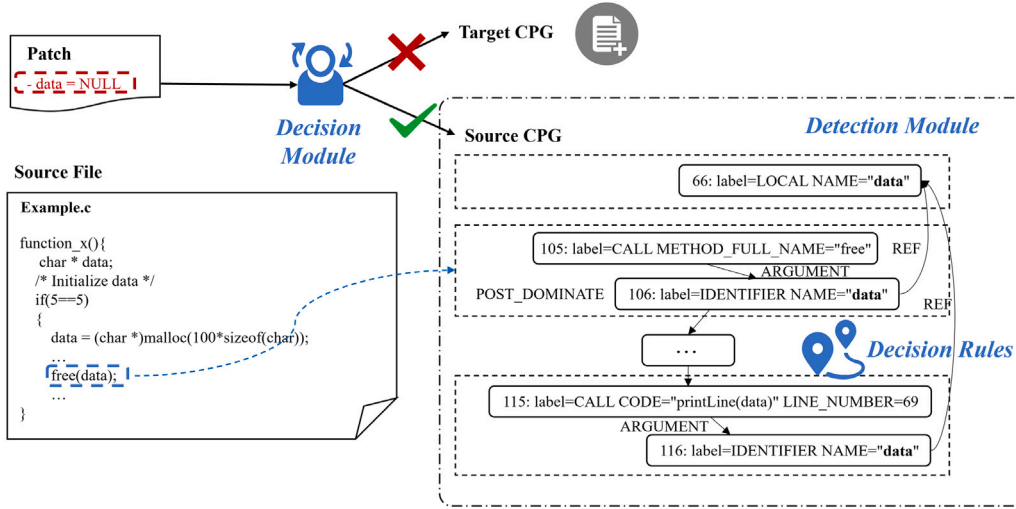


Fig. 3. An illustration of the CAPRA detection methodology in addressing a UAF vulnerability. CAPRA evaluates the patch incrementally, parsing it line by line. Within this process, the decision module identifies the next set of analysis files and pertinent detection rules, contingent upon whether the current line involves an addition or a deletion. This approach assesses if the changes made by the patch might induce an immature vulnerability, following the predefined criteria.

3.2. Detection scripts

The detection scripts represent another crucial design component, with each script focusing on a distinct category of software vulnerability. All detection scripts consist of two core modules: a decision module and detection rules. The decision module invokes appropriate detection rules based on the patch modification type (addition or deletion) and the modification content. The detection module conducts targeted risk assessments, allowing for the coexistence of multiple detection rules within a single script to tackle diverse risk scenarios.

Both the decision-making module and detection rules are designed based on fundamental principles of vulnerability formation. To more clearly illustrate CAPRA's detection process, Fig. 3 shows a detection example for UAF vulnerability. In this example, when dynamically allocated memory is freed but its corresponding pointer is not nullified and continues to be used later in the code, a UAF vulnerability occurs.

Upon identification by CAPRA that a patch involves the removal of statements assigning NULL, the *Decision Module* forwards the removed line alongside the source file's CPG to the *Detection Module*. The *Detection Module* then assesses whether this removal satisfies the conditions necessary to activate a UAF vulnerability, considering both the contextual details of the line and the predefined *Detection Rules*.

3.3. High-level architecture of CAPRA

Algorithm 1 encapsulates the aforementioned process in three distinct steps: patch content preprocessing, code property graph generation, and patch risk detection. Serving as the high-level architecture of CAPRA, the algorithm takes as input the patch file P under review and the corresponding pre-patched file F_{pre} . The output R comprises the risk analysis results, including information on potentially risky lines and their associated risk descriptions.

Before presenting the algorithm, we define the notation used: “ $\leftarrow$ ” denotes assignment (e.g., “ $x \leftarrow y$ ” means “ x is assigned the value of y ”), “ $\emptyset$ ” represents an empty set, “ $\in$ ” indicates set membership (e.g., “ $x \in S$ ” means “ x is an element of set S ”), “ $\cup$ ” denotes set union, and “ $\cap$ ” represents set intersection, superscript “ c ” denotes the complement of a set.

Algorithm 1 Patch Security Assessment Using CAPRA

Input: A patch file P under review; a set of pre-patched files F_{pre} corresponding to P

Output: A set R of potentially risky lines in P and associated risk summaries

- 1: $R \leftarrow \emptyset$ {result set of detection outcomes}
- 2: Applying patch P to F_{pre} to obtain the modified file F_{post}
- 3: Parse the detailed modification information for each file in P and store it in set Set_P
- 4: $lines_{added}, lines_{removed} \leftarrow \emptyset$ {record the deletions and additions for each patched file}
- 5: **for** each patched file $f_{patched} \in Set_P$ **do**
- 6: **if** complete file deletion or addition **then**
- 7: continue
- 8: **else**
- 9: **for** each hunk $h \in f_{patched}$ **do**
- 10: Extract line's data li_{added} from hunk h_{post} in F_{post}
- 11: Extract line's data $li_{removed}$ from hunk h_{pre} in F_{pre}
- 12: **end for**
- 13: **end if**
- 14: $lines_{added}[f_{patched}] \leftarrow lines_{added} \cup li_{added}$
- 15: $lines_{removed}[f_{patched}] \leftarrow lines_{removed} \cup li_{removed}$
- 16: **end for**
- 17: Generate the code property graph CPG_{source} by parsing F_{pre}
- 18: Generate the code property graph CPG_{target} by parsing F_{post}
- 19: Create a set $Set_{instance}$ of detection instances based on detection scripts, where each instance corresponds to a specific vulnerability type
- 20: **for** each instance $i \in Set_{instance}$ **do**
- 21: **for** each added line $l_{added} \in lines_{added}$ **do**
- 22: $R_{l,j} \leftarrow \text{analysis } l_{added} \text{ with } i \text{ and } CPG_{target}$
- 23: $R \leftarrow R_{l,j} \cup R$
- 24: **end for**
- 25: **for** each removed line $l_{removed} \in lines_{removed}$ **do**
- 26: $R_{l,j} \leftarrow \text{analysis } l_{removed} \text{ with } i \text{ and } CPG_{source}$
- 27: $R \leftarrow R_{l,j} \cup R$
- 28: **end for**
- 29: **end for**
- 30: **return** R

Step 1. Preprocessing of patch content (refer to lines 3–15 in Algorithm 1): Initially, the modification information is extracted from the patch file and organized according to file names and types of modifications. Several crucial aspects of this process need consideration: Firstly, it is necessary to assess whether a complete file is removed or added, given that the emphasis is on security issues within patches.

Another significant facet is the management of comments. Comments that do not influence the logical structure of the code are removed. Nonetheless, if substantial code is effectively rendered inactive through the addition of comments, these modifications are treated as deletions for the purpose of further analysis.

Step 2. CPG generation (refer to lines 16–17 in Algorithm 1). We generate two code property graphs (CPG_{source} and CPG_{target}) by analyzing both the pre-patched file (F_{pre}) and post-patched file (F_{post}). The resulting graph representations are then consolidated into a single code property graph.

Step 3. Patch risk detection (refer to lines 18–30 in Algorithm 1). This step executes the core detection process. We begin by initializing detection scripts for each vulnerability type, then separately analyze the added and removed lines. Within these detection scripts, decision modules implement specific detection rules contingent on the nature of the input line. The detailed description of the method will be elaborated in Section 4. Upon identifying a security vulnerability within a scrutinized line, pertinent data – such as the line number, type of vulnerability, and a concise risk summary – are compiled into the result set R . This methodology facilitates the expedited detection of security risks and the generation of precise recommendations for code reviewers.

4. Methodology

This section details CAPRA's methodology for identifying security vulnerabilities introduced during code modifications, focusing specifically on our detection mechanisms for memory leak and UAF vulnerabilities.

4.1. Memory leak detection

Memory leak constitute a significant vulnerability class in C/C++ applications. Analysis of the CVE database reveals that such vulnerabilities represented over 23% of all memory-related security issues among the top 1000 C/C++ GitHub projects between 2020 and 2023.

Effective code review practices combined with static analysis tools can reduce memory leak incidents by approximately 55%. Given this significant prevention potential, along with the economic impact described previously, CAPRA prioritizes memory leak as a primary detection target.

4.1.1. Vulnerability mechanism

Memory leak fundamentally occurs when programs fail to deallocate dynamically allocated memory, leading to progressive system resource depletion. There are three primary scenarios that precipitate memory leak:

I. **Forgotten Deallocation:** This most common memory leak scenario occurs when code dynamically allocates memory but fails to include corresponding deallocation statements. Recent OpenSSH fix commit (Larrosa, 2024) exemplify this vulnerability pattern.

II. **Pointer Reassignment Leaks:** This scenario occurs when a pointer that already references allocated memory is reassigned before the original allocation is freed.

III. **Incorrect Memory Management:** This category encompasses subtle logical errors in complex data structures. For example, a common mistake in linked list implementation where only the head node is deallocated, while memory allocated for subsequent nodes remains unreleased.

4.1.2. Script detection logic

Memory leaks triggered by patches manifest in two primary manners. Initially, such leaks may occur upon the removal of essential deallocation statements from pre-existing code. Additionally, they may arise when new memory allocation operations are introduced without the necessary deallocation statements. CAPRA is designed to specifically focus on these modifications at the patch level to detect potential memory leaks before they are deployed in production code.

Based on these patterns, CAPRA implements a targeted analysis approach: *when examining patch modifications, the system analyzes source files when detecting deleted memory deallocation operations, and analyzes target files when identifying newly added memory allocation operations*. This dual-focused strategy ensures comprehensive detection of potential memory leak vulnerabilities introduced during code changes.

CAPRA's detection rules center on contextual memory management analysis. When the system identifies memory allocation or deallocation operations in a patch, it evaluates whether proper memory management practices are maintained within the relevant scope. Algorithm 2 details this analytical process, which systematically identifies potential memory leak introduced by code modifications.

Algorithm 2 Detecting memory leak in Single-Line Code Modifications

Input: Code line l to be analyzed; Code property graph cpg ; File type $type$ (SOURCE for pre-patch or TARGET for post-patch); A set Set_n of corresponding nodes for l in cpg

Output: Memory leak detection results for l

```

1: if  $type$  is SOURCE (analyzing a deleted line) then
2:   Retrieve variable  $v$  and node  $n$  containing memory deallocation
   operation in  $l$ 
3:   if No memory deallocation-related operation present in  $l$  then
4:     return Deletion of  $l$  will not result in memory leak
5:   else
6:     Retrieve the set of memory allocation nodes  $Set_{alloc_v}$  associated
     with  $v$  in the context
7:     Retrieve the set of memory deallocation nodes  $Set_{dealloc_v}$  asso-
     ciated with  $v$  in the context
8:     if  $Set_{dealloc_v}$  is empty then
9:       return Memory leak risk detected in deletion of  $l$  due to
       unreleased memory for  $v$ 
10:    else if  $Set_{alloc_v}$  is not empty then
11:      Retrieve nodes  $nodes_{risk}$  from  $Set_{alloc_v}$  that have not been
      deallocated in  $Set_{dealloc_v}$ 
12:      if  $nodes_{risk}$  is empty then
13:        return Deletion of  $l$  will not result in memory leak
14:      else if memory corresponding to  $nodes_{risk}$  can be deallocated
      within  $Set_n$  then
15:        return Memory leak risk detected in deletion of  $l$ ; atten-
        tion required for lines corresponding to  $nodes_{risk}$ 
16:      else
17:        return Deletion of  $l$  will not result in memory leak. How-
        ever, variables in lines corresponding to  $nodes_{risk}$  present
        potential deallocation risks
18:      end if
19:    else
20:      return Deletion of  $l$  will not result in memory leak
21:    end if
22:  end if
23: else if  $type$  is TARGET (analyzing an added line) then
24:   Retrieve variable  $v$  and node  $n$  containing memory deallocation
   operation in  $l$ 
25:   if No memory allocation-related operation present in  $l$  then
26:     if  $l$  contains  $v$ , and there exists a corresponding memory allo-
     cation operation node  $n_{alloc}$  in the preceding context then
27:        $n = n_{alloc}$ 
28:     else
```

```

29:     return Addition of  $l$  will not result in memory leak
30: end if
31: else
32:   Retrieve the set of memory allocation nodes  $Set_{alloc_v}$  associated
   with  $v$  in the context
33:   Retrieve the set of memory deallocation nodes  $Set_{dealloc_v}$  asso-
   ciated with  $v$  in the context
34:   if  $Set_{dealloc_v}$  is empty then
35:     return Memory leak risk detected in the addition of  $l$  due
     to absence of corresponding variable deallocation operations
     in the context
36:   else if all variables in  $Set_{alloc_v}$  are successfully deallocated
   within nodes of  $Set_{dealloc_v}$  then
37:     return Addition of  $l$  will not result in memory leak
38:   else
39:     return Potential memory leak risk detected in the addition
     of  $l$ , as memory allocated to  $v$  remains unreleased
40:   end if
41: end if
42: end if

```

Our detection method features a dual-branch analysis approach that examines both code deletions and additions. The algorithm processes a target line of code and evaluates it for potential memory leak vulnerabilities. Specifically, the algorithm requires four inputs: the line l of code under examination, the code property graph cpg derived from the patch file, the file type, and the set of nodes Set_n that correspond to the target line within the graph structure. After completing its analysis, the algorithm produces a comprehensive assessment of memory leak risks associated with the examined line.

Part 1 (refer to lines 1–22 in Algorithm 2): When the input type is ‘SOURCE’, indicating l is being removed, the decision mechanism needs to determine whether l contains memory deallocation operations and retrieve the deallocated memory’s variable v and node n (line 2). Typical memory deallocation operations include direct calls to functions such as `free()`, `kfree()`, as well as non-standard library functions known to deallocate parameter variables’ memory in vulnerability databases, such as `nft_pipapo_insert()` and `nft_pipapo_remove()` (NVD, 2023). These functions are documented in an XML file for easy extension.

We also need to consider memory deallocation operations performed through indirect function calls that have appeared in previously disclosed vulnerabilities. For more details on the analysis algorithm for indirect memory deallocation operations through function calls, please refer to Section 4.1.3.

If memory deallocation operations exist in l , we search the context using data flow and control flow analysis to identify the set of memory allocation nodes Set_{alloc_v} and the set of memory deallocation nodes $Set_{dealloc_v}$ related to v (lines 6–7 in Algorithm 2). First, we traverse cpg and record nodes containing memory allocation and deallocation operations along with their corresponding variables: if the current node’s label is ‘CALL’, indicating a function call operation, we can locate nodes with ‘IDENTIFIER’ labels in its subsequent ‘ARGUMENT’ edges. Second, we compute the equivalence relationships between v in l and the aforementioned variables.

If $Set_{dealloc_v}$ is empty, indicating that v ’s memory is not deallocated in the subsequent context, the deletion of l may pose a memory leak risk. It should be noted that false positives may occur when the patch simultaneously removes the memory allocation operation for v . However, given that such false positives are relatively easy to identify, we opt to leave the decision-making authority to the reviewer.

If $Set_{dealloc_v}$ contains elements, we analyze whether all variables in Set_{alloc_v} can be deallocated through nodes in $Set_{dealloc_v}$ (line 11 in Algorithm 2).

To reduce computational overhead, we first extract a subgraph from cpg that contains only data flow and control flow. Then we determine if paths exist between nodes in Set_{alloc_v} and $Set_{dealloc_v}$. To mitigate

hypocrite commit-style attacks, paths containing decision nodes in their control flow are also classified as non-existent.

Path computation varies depending on the tool used to generate the code property graph. In this paper, we utilize Joern 2.0.4, where data path edge labels include ‘ALIAS’, ‘REACHING_DEF’, ‘REF’, and ‘ARGUMENT’. Variable addresses can be passed through parameters and return values during function calls. Therefore, we also consider ‘CALL’ and ‘RETURN’ control relationships. While This approach likely increases the false positive rate, CAPRA is fundamentally designed to assist patch security review by providing risk alerts, with final determinations remaining with the reviewers. Therefore, we consider this trade-off acceptable.

When variables with improperly deallocated memory exist and their memory can be deallocated within Set_n , the deletion of l is considered to pose a memory leak risk, necessitating reviewer attention to the lines containing nodes $nodes_{risk}$ corresponding to these unreleased variables.

Part 2 (refer to lines 23–41 in Algorithm 2): When the input type is ‘TARGET’, indicating that line l is newly added in this modification, we focus on memory allocation operations. Similar to the logic in Part 1, we first determine whether direct or indirect memory allocation operations exist in line l , and retrieve the variable v and node n associated with the allocated memory.

We additionally consider a scenario where v in the source program has been dynamically allocated memory but not actively deallocated in the subsequent code, indicating an existing memory leak. In this case, if l contains operations related to v , the detection script performs forward analysis to obtain information about the node where v was allocated memory in the preceding context.

If a memory allocation operation exists in the newly added line, the sets of memory allocation nodes Set_{alloc_v} and memory deallocation nodes $Set_{dealloc_v}$ related to v are recorded. The calculation method differs from Part 1 in one aspect: in Part 1, memory deallocation nodes within indirect functions called by l are not counted since this code logic will be deleted along with patch integration, whereas in Part 2, indirect deallocations are also included in $Set_{dealloc_v}$.

If the final recorded $Set_{dealloc_v}$ is an empty set, indicating that memory allocated to v is not promptly deallocated in the context, a memory leak risk exists. This detection logic carries a risk of false positives: Our contextual analysis is limited in scope, focusing only on files related to the current patch modification, while v ’s memory might be deallocated in other files. However, based on our years of development experience, the probability of such scenarios is relatively low, and deallocating memory within a limited scope after allocation represents good secure coding practice.

When all memory for variables in Set_{alloc_v} is successfully deallocated within nodes of $Set_{dealloc_v}$ (line 32 in Algorithm 2), the addition of l is considered free of memory leak risks; otherwise, a warning is generated indicating that v is not properly deallocated following dynamic memory allocation.

4.1.3. Indirect function call analysis

In this paper, indirect memory operations primarily refer to memory management operations performed through indirect function calls, mainly including cross-function boundary memory management, standard memory operations encapsulated through custom interfaces, and function return value transmission. Through Fig. 4, we provide a simple explanation of direct memory management and memory management through indirect function calls.

In Fig. 4, variable `directBuffer` is allocated memory in `example()` (line 3) and freed within the same function by calling the standard API `free()` (line 4). This contextually clear operation is defined as direct memory management. The variable `indirectBufferA` is allocated memory in line 6 of the current function and subsequently passed as a parameter to the `indirect_operation()` function (line 8). This function releases the input memory in line 3, thereby indirectly freeing the memory of `indirectBufferA` in `example()`. The variable `indirectBufferB`

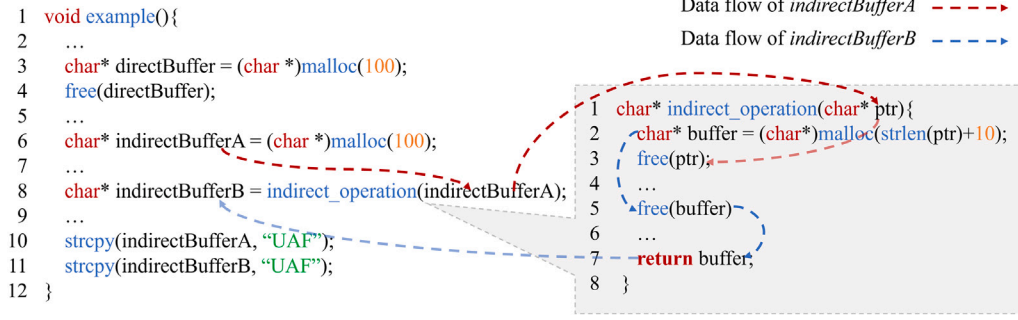


Fig. 4. An example of memory deallocation through direct and indirect function calls. The red dotted line describes how *indirectBufferA* implements indirect memory deallocation through cross-function boundary memory management and custom interface encapsulation. The blue dotted line illustrates how *indirectBufferB* is assigned deallocated memory through function return value transmission. Both approaches may pose UAF risks.

receives the return value *buffer* from the *indirect_operation()* function. Variable *buffer* is allocated memory in line 2 of *indirect_operation()*, but after use, it is freed in line 5. Consequently, *indirectBufferB* will point to a segment of already deallocated memory. Ultimately, calling *indirect_operation()* will result in UAF vulnerabilities when subsequently using the memory of *indirectBufferA* and *indirectBufferB*.

For developers and code reviewers, changes in variables during indirect function calls are easily overlooked. This lack of contextual information is also a direct cause of many triggered immature vulnerabilities.

In CAPRA's analysis process, the challenge of indirect memory operations lies in correctly identifying whether these operations will trigger vulnerabilities such as memory leaks or UAF. Analysis based solely on direct function calls is insufficient for this purpose. To address the aforementioned problems, we will identify and track memory deallocation behaviors affected by indirect function calls, as shown in Algorithm 3.

Algorithm 3 Detection of Dynamic Memory Deallocation Through Indirect Function Calls

Input: Code property graph *cpg*; A set Set_n of corresponding nodes for *l* in *cpg*

Output: Nodes $nodes_{indirect}$ where memory is deallocated within functions and requires tracking

```

1: Initialize  $nodes_{indirect} \leftarrow \emptyset$ 
2:  $func_{name} \leftarrow$  name of the called function from  $Set_n$ 
3: Locate the function definition node  $func_{node}$  and all nodes contained within the function  $Set_{func}$  by using  $func_{name}$  and function call edges in cpg
4: Record the function input parameters  $nodes_{method\_in}$  and the function return value  $nodes_{method\_out}$ 
5: for each  $node_{current} \in Set_{func}$  do
6:   if  $node_{current}$  is a memory deallocation operation node then
7:     if Memory pointed to by  $nodes_{method\_in}$  is deallocated at  $node_{current}$  then
8:       Add corresponding input parameter nodes and tracking markers to  $nodes_{indirect}$ 
9:   end if
10:  if The pointer that deallocates memory in  $node_{current}$  is returned at  $nodes_{method\_out}$  then
11:    Add corresponding return value nodes and tracking markers to  $nodes_{indirect}$ 
12:  end if
13:  if  $node_{current}$  is a function indirect call that contains input parameters or return value then
14:     $nodes_{indirect} = \text{Algorithm 3}(cpg, node_{current})$ 
15:  end if
16: end if
17: end for

```

18: return $nodes_{indirect}$

Algorithm 3 takes two inputs: the code property graph *cpg* and the set of nodes Set_n corresponding to line *l*. It outputs the set of nodes $nodes_{indirect}$ where memory is deallocated within the function.

When analyzing a code line with an indirect function call, we follow these steps:

Step 1. Parse the called function name $func_{name}$.

Step 2. Find the function definition location $func_{node}$ using function's name and call edges in *cpg*.

Step 3. Obtain all nodes Set_{func} by analyzing 'CONTAINS' edges (in the code property graph, the starting node of a function is connected to the internal nodes within the function through edges of type 'CONTAINS').

Step 4. Traverse Set_{func} to find memory deallocation nodes.

Step 5. For each deallocation node, evaluate its relationship with function input parameters ($nodes_{method_in}$) and return values ($nodes_{method_out}$).

Step 6. If the current node is another indirect function call, recursively apply Algorithm 3 to check for memory deallocation of parameters or return pointers (lines 6–16).

Variables are considered equivalent if they satisfy any of the following four relationships:

- Input variables v_1 and v_2 represent the same local variable.
- v_1 and v_2 represent the same global variable.
- The memory address of v_2 is indirectly passed to v_1 through function calls.
- v_1 and v_2 maintain an alias relationship.

The main logic of the computation of variable alias is that traverse successor nodes of starting node. If a memory transfer path exists between a certain successor node and the starting node, update the alias set and use it as a new starting node for iterative computation. Furthermore, it is crucial to omit any implicit connections associated with standard library functions, such as *printLine()*, from the CPG.

CAPRA enhances efficiency by examining solely the patch-modified files, even though the iterative nature of our algorithm introduces some additional computational expenses. Such a focused strategy substantially diminishes the computational burden, thereby facilitating a more lightweight deployment suited for continuous integration environments.

4.2. Use-After-Free detection

C/C++ and other memory-unsafe programming languages remain fundamental to critical systems development, including kernels and browsers, despite generating thousands of memory safety vulnerabilities annually. UAF vulnerabilities – a temporal memory error occurring when programs access previously deallocated memory – represent one of the most prevalent memory safety flaws. Researches of the CVE database reveals that UAF vulnerabilities constitute an increasing proportion of severe security issues, highlighting the need for enhanced detection methods (Wickman et al., 2021).

4.2.1. Vulnerability mechanism

Vulnerabilities associated with UAF are fundamentally linked to the mechanisms of dynamic memory allocation. Dynamic memory, or the heap, allows programmers to allocate blocks of arbitrary sizes, which can subsequently be altered or deallocated.

Programs access data in dynamic memory through pointers. A dangling pointer occurs when memory is deallocated or relocated without updating the corresponding pointer references. When these dangling pointers are subsequently dereferenced, they access either invalid memory locations or, more critically, memory regions that have been reallocated to new objects.

There are two typical cases of UAF vulnerabilities. The most common UAF vulnerability case is that a buffer is deallocated but without nullifying the buffer pointer. Consequently, the buffer in the global variable becomes a dangling pointer, and the UAF vulnerability is triggered when the pointer is subsequently called. A more implicit triggering mechanism is reference counting in a multi-threaded environment, such as CVE-2019-12819.

4.2.2. Script detection logic

To trigger a UAF vulnerability during patch integration, one of two conditions must be met: either introducing code that manipulates dangling pointers referencing deallocated memory, or removing pointer nullification operations where the affected memory is subsequently accessed. Our detection algorithm therefore implements the following decision logic: *If an added line contains memory deallocation or memory access operation, analyze the target file; if a removed line contains pointer nullification operation, analyze the source file.*

Our detection rules focus primarily on analyzing pointer operations and memory access patterns following deallocation events. This approach enables identification of potential UAF vulnerabilities by tracking the relationship between memory management operations and subsequent pointer usage. The detailed algorithmic implementation is presented in Algorithm 4.

Algorithm 4 Detection of Use-After-Free Vulnerabilities in Single-Line Code Modifications

Input: Code line l to be analyzed; Code property graph cpg ; File type $type$ (SOURCE for pre-patch or TARGET for post-patch); A set Set_n of corresponding nodes for l in cpg

Output: UAF detection results for l

```

1: Initialize  $cpg_{light}$  as a subgraph representing data flow and control
  flow from  $cpg$ 
2: if  $type$  is TARGET (analyzing an added line) then
3:   Retrieve variable  $v$  and node  $n$  containing memory deallocation
  operation in  $l$ 
4:   if  $v$  is not NULL and  $n$  is not NULL then
5:      $Set_{uses} \leftarrow$  the Set of successive usage paths for  $v$  in  $cpg_{light}$ 
6:     if  $Set_{uses}$  is empty then
7:       return The addition of  $l$  potentially introduces UAF risk,
        where  $v$  becomes dangling after its referenced memory is
        deallocated
8:     else if There exist paths in  $Set_{uses}$  where memory access occurs
      without prior pointer nullification then
9:       return The addition of  $l$  leads to a UAF vulnerability,
        where  $v$  becomes dangling after memory deallocation and
        is subsequently accessed
10:    end if
11:  end if
12:  Retrieve variable  $v$  and node  $n$  containing memory access operation
  in  $l$ 
13:  if  $v$  is not NULL and  $n$  is not NULL then
14:    Retrieve the set of memory deallocation nodes  $Set_{dealloc_v}$  asso-
    ciated with  $v$  in the context
15:    Retrieve the set of nullification nodes  $Set_{nullify}$  from the suc-
    cessor nodes of  $Set_{dealloc_v}$ 

```

```

16:  for  $n_{dealloc} \in Set_{dealloc_v}$  do
17:    if There exists paths from  $n_{dealloc}$  to node  $n$  without any
      nullification operations on  $v$  along the paths then
18:      return The addition of  $l$  results in a UAF vulnerability, as
        the memory accessed by  $v$  has already been deallocated at
        the line corresponding to  $n_{dealloc}$ 
19:    end if
20:  end for
21: end if
22: else if  $type$  is SOURCE (analyzing a deleted line) then
23:  Retrieve variable  $v$  and node  $n$  containing nullification operation
  in  $l$ 
24:  if  $v$  is not NULL and  $n$  is not NULL then
25:    if The memory pointed to by  $v$  has not been deallocated in the
      preceding context then
26:      return No UAF risk is detected in the addition of  $l$ 
27:    end if
28:     $Set_{uses} \leftarrow$  the Set of successive usage paths for  $v$  in  $cpg_{light}$ 
29:     $Set_{uses} \leftarrow Set_{uses} \cap (Set_n)^c$ 
30:    if  $Set_{uses}$  is empty then
31:      return The deletion of  $l$  potentially introduces UAF risk,
        where the corresponding  $v$  becomes dangling
32:    else if There exist paths in  $Set_{uses}$  where memory access occurs
      without prior pointer nullification then
33:      return The deletion of  $l$  leads to a UAF vulnerability,
        where  $v$  becomes dangling after memory deallocation and
        is subsequently accessed
34:    end if
35:  end if
36: end if
37: return No risk of UAF triggering was found in  $l$  at present.

```

Similar to Algorithm 2, Algorithm 4 encompasses two branches: analysis of code deletion operations and analysis of code addition operations. The algorithm similarly takes as input the line l under detection, the code property graph cpg of the file containing the patch under detection, the file type $type$, and Set_n , the set of nodes corresponding to l in cpg .

As introduced in Section 2.3 and Fig. 2, CPG represents a multiple graph structure that simultaneously integrates abstract syntax tree, data flow graph, and control flow graph. For our UAF vulnerability detection algorithm, we discovered that only data flow and control flow information are essential. Therefore, Algorithm 4 first prunes the complete CPG, preserving only the subgraph cpg_{light} containing data flow and control flow relationships. This strategy effectively reduces additional memory consumption when loading complete CPG for complex files and decreases computational overhead during static flow analysis using traversal algorithms.

Part 1 (refer to lines 2–21 in Algorithm 4): When the input type is ‘TARGET’, indicating that l is to be added, analysis is required for memory deallocation operations and memory access operations within l .

For memory deallocation operations, we attempt to parse variable v and node n corresponding to the memory deallocation from l (line 3).

If l contains memory deallocation operation (including explicit and indirect memory deallocation operations as described in Algorithm 3), further analysis is conducted on the subsequent usage patterns of the variable, tracking and collecting the set of successive usage paths (Set_{uses}) for v from cpg_{light} (line 5). When v is a function parameter or an alias of a parameter of the current function, Set_{uses} must also include the successors of external call sites. The script employs a depth-first traversal strategy, initiating from n , attempting to locate nodes labeled as ‘IDENTIFIER’ and indirect function call nodes of equivalent variables.

If Set_{uses} is empty, indicating that the variable remains unused within the finite subsequent context (also implying that the corresponding pointer becomes dangling after memory deallocation), although

this does not directly constitute a UAF vulnerability, reviewers will be notified about the potential risk of an incipient UAF vulnerability.

If Set_{uses} is non-empty, it means the deallocated memory is accessed later. In this case, we examine pointer nullification operations (line 8). We check Set_{uses} for any nullification operations before memory access occurs. If memory is accessed without prior nullification, a UAF vulnerability exists. If nullification occurs first, the code is secure.

It should be noted that we must examine the control dependencies between conditional nodes and nullification operations, and consider nullification operations with such control dependencies as invalid. This design is intended to address hypocrite commit attacks where nullification operations become unreachable through conditional statements. Adding processing logic for conditional nodes undoubtedly increases the false positive rate, as we currently cannot achieve the precision of reachability analysis comparable to symbolic execution methods.

If l contains no memory deallocation operations, further analysis is conducted to determine the presence of memory access operations (lines 12–21). The operated variable v and its corresponding node n are parsed from l , followed by searching for dynamic memory deallocation nodes in cpg , and identifying nodes corresponding to the same memory as v through construction of a temporary reverse graph, denoted as $Set_{dealloc_v}$. Subsequently, from the successor nodes of $Set_{dealloc_v}$, we compute the set of nullification operation nodes for v , denoted as $Set_{nullify}$.

Based on UAF formation mechanism, we determine in cpg_{light} whether $Set_{nullify}$ nodes lie on the path between the nodes in $Set_{dealloc_v}$ and node n . To eliminate false positives caused by local variables with the same name, the edges labeled as ‘SOURCE_FILE’ in the CPG must be masked when computing paths.

We mask ‘SOURCE_FILE’ edges during path computation because in the CPG, these edges connect nodes to their respective source files (Fabian et al., 2023). If these edges were not excluded during variable reachability analysis, the algorithm would erroneously establish data flow or control flow relationships between different variables solely based on their presence within the same source file. This masking operation is implemented by specifically excluding edges of ‘SOURCE_FILE’ type when constructing traversal paths, ensuring our analysis considers only genuine data dependencies and control dependencies. This approach significantly reduces false positive detections, particularly when analyzing complex codebases where numerous variables may share the same source file without having actual dependencies between them.

A UAF vulnerability is considered triggered if no nullification operations exist between the memory deallocation operation and n , otherwise, a null pointer dereference risk is flagged (while null pointer dereference detection is not the primary focus of this script).

The path computation process also takes into account the impact of conditional logic.

Part 2 (refer to lines 22–33 in Algorithm 4): When the input type is ‘SOURCE’, indicating that l is to be deleted, analysis of nullification operations within l is required.

Firstly, we parse pointer variable v that undergoes nullification and its corresponding node n in the CPG. Making ensure that v has not been deallocated in the preceding context. In identifying nullification operations, beyond direct operations like “ $v=NULL$ ”, we also consider functions containing nullification operations that frequently appear in publicly disclosed vulnerabilities, such as `release()`, `delete_if_unused()`.

Subsequently, we compute the set of successive paths Set_{uses} for v in cpg_{light} , and remove its intersection with Set_n , as we are analyzing pre-patched files, necessitating the elimination of nullification operations contained within l . As in Part 1, when v is a function parameter or an alias of a parameter of the current function, Set_{uses} must also include the successors of external call sites.

Since vulnerabilities associated with memory frequently extend beyond individual function boundaries, it is crucial to monitor variable usage through the course of function calls. As elaborated in

Section 4.1.2, memory operations initiated through indirect function calls can establish vulnerability chains involving multiple functions. Consequently, our analysis of successive usage paths incorporates external call sites related to function parameters and their aliases. This approach to cross-functional tracking is vital to thoroughly identify vulnerabilities that emerge via parameter transmission and function return processes. Without such comprehensive tracking, numerous complex vulnerabilities could evade detection.

If Set_{uses} is empty, indicating that the patch has removed the sole nullification operation for v within the finite context, consequently, a potential UAF vulnerability risk is flagged. The current detection logic may generate false positives when encountering patches that simultaneously remove both memory deallocation and subsequent operation code, we opt to defer such straightforward scenarios to manual reviewer assessment.

If Set_{uses} contains paths where memory access occurs without prior nullification, the deletion of l is considered to trigger a UAF vulnerability, reviewers are notified and the lines with potential UAF vulnerability risks are marked for identification.

5. Experimental evaluation

We present the experimental data and test environment for validating CAPRA, including our key evaluation metrics. We then demonstrate CAPRA’s detection logic and analytical workflow through representative case studies.

5.1. Experimental setup

5.1.1. Experimental dataset

CAPRA identifies potential triggers of immature vulnerabilities rather than focusing on detection accuracy, which complicated the selection of suitable sample repositories from mainstream datasets. Therefore, we constructed a dedicated dataset for the purpose of validating CAPRA. We selected 100 memory leak and UAF vulnerability samples from SARD as baseline data and constructed 400 test cases. To analyze CAPRA’s false positive rate in real-world scenarios, we constructed a dataset RealWorld comprising approximately 120 samples of legitimate commits from multiple open-source projects including Linux, OpenSSH, ffmpeg, and others. Additionally, the dataset incorporates hypocrite commit samples and documented vulnerabilities triggered by legitimate commits, including CVE-2019-12819.

For each SARD base case, we created four test categories: vulnerability-triggering code additions, vulnerability-triggering code deletions, safe code additions, and safe code deletions. These test cases simulate realistic scenarios where developers inadvertently introduce vulnerabilities through both simple and complex code changes.

Our experimental dataset² and source code have been uploaded to the CAPRA’s git repository, where researchers can access the dataset and reproduce our experimental results.

5.1.2. Tool implementation and experimental environment

CAPRA operates on Python 3.10 in an Ubuntu 22.04.4 LTS environment with 8 CPU cores and 8 GB memory. Dependencies include Joern 2.0.441 for code property graph generation and NetworkX 3.3 for graph analysis.

5.2. Case study

We analyze three representative cases, including real-world vulnerability and hypocrite commit attack. Each analysis presents the case origin and defect type, followed by CAPRA’s detection logic and results.

² <https://github.com/TangBX-wtu/CAPRA/tree/main/DataSet-2025>

```

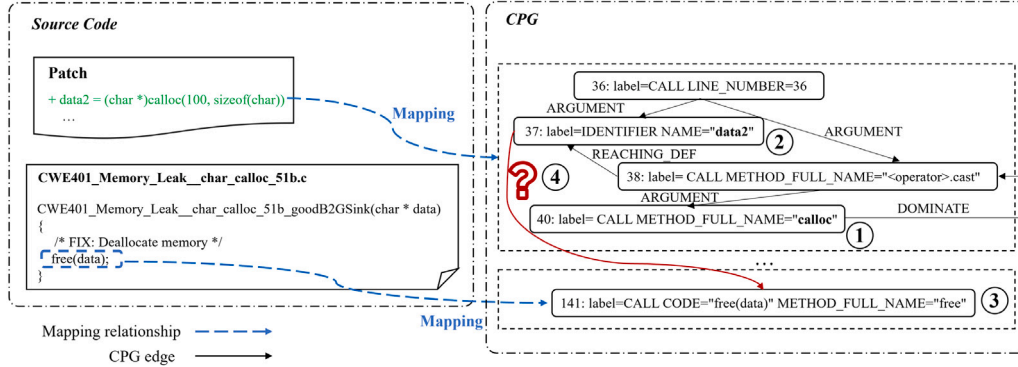
void CWE401_Memory_Leak__char_calloc_51_bad()
{
    char * data;
    data = NULL;
    /* POTENTIAL FLAW: Allocate memory on the heap */
    data = (char *)calloc(100, sizeof(char));
    /* Initialize and make use of data */
    strcpy(data, "A String");
    printLine(data);
    CWE401_Memory_Leak__char_calloc_51b_badSink(data);
}

void CWE401_Memory_Leak__char_calloc_51b_badSink(char * data)
{
    /* POTENTIAL FLAW: No deallocation */
    /* empty statement needed for some flow variants */
}

```

(a) The source code of the *bad()* method (b) The source code of the *badSink()* method

Fig. 5. Code example of memory leak for 99230-v1.0.0.

Fig. 6. Memory leak detection and analysis process for Case Study 1. The submitted patch dynamically allocates memory for *data2*, and CAPRA determines memory leak risk by analyzing whether there are matching memory release operations for the same address in the subsequent code.

5.2.1. Case study 1

Case 1 examines a memory leak from SARD 2022-08-11-juliet v1.3.1 (case 99230-v1.0.0), demonstrating cross-file memory management. In CWE401_51a.c, the variable *data* is initialized in the *bad()* method and passed to *badSink()* in CWE401_51b.c, where it is not deallocated after use (Fig. 5).

The experimental patch simulated memory leak conditions by exploiting code reviewer's limited comprehension of indirect function logic. The implementation initialized a new variable to mirror the operational sequence of the variable *data* in Fig. 5.

CAPRA's initial phase comprises general analysis (Section 3.3), which executes through three sequential operations: (1) construction of code property graphs for source and target files; (2) parsing of the patch to extract line numbers and content modifications; and (3) transmission of granular modification data to the detection scripts.

The second phase identifies memory leak vulnerability triggers (Algorithm 2). For code additions in the initial modification, the detection script loads the target file's corresponding CPG according to predefined analytical parameters. Fig. 6 illustrates the defect trigger analysis process for this modification.

Fig. 6 juxtaposes the initial modified code line and partial source program with the CPG's critical nodes. In this figure, the upper left shows the code line in the current patch that triggers immature vulnerability, while the bottom half displays key code segments involved in the subsequent analysis process. The two dashed boxes on the right (upper and lower) represent the corresponding nodes in the code property graph for the patch and key code respectively (one line of code may map to multiple nodes). The blue dotted lines in Fig. 6 represent the mapping from code lines to CPG nodes, the black solid lines represent the connection relationships between nodes in the CPG, and the numbered markers indicate the key steps in CAPRA detection.

The analysis proceeds through four sequential steps:

Step 1. CAPRA extracts nodes 37–40 from the CPG corresponding to the target line addition, where node 40's label ('label=CALL' indicates a function call) and property ('METHOD_FULL_NAME') indicate dynamic memory allocation operations ('calloc').

Step 2. Through analysis of 'IDENTIFIER' category label (indicates the manipulation relationship of variable addresses) and internodal control flow relationships, CAPRA identifies the memory operation object *data2* and its corresponding node 37.

Step 3. CAPRA filters memory deallocation operation nodes within the current CPG, exemplified by node 141, which corresponds to *free(data)*; in the *goodB2GSink()* method located in a separate file CWE401_51b.c.

Step 4. Through comprehensive analysis of CPG data and control flows, CAPRA determines that node 37's *data2* lacks viable deallocation paths through node 141's corresponding code. Since node 141 is the only node in the current context that can perform memory deallocation operations, CAPRA will determine that the memory allocated for *data2* cannot be freed in the subsequent context, indicating a memory leak risk.

We analyze subsequent patch modifications similarly, skipping lines without potential defects to optimize efficiency. CAPRA synthesizes the results of the analysis to produce a comprehensive vulnerability assessment, describing the modifications of high risk and their corresponding technical rationales.

In this case, CAPRA will identify a potential memory leak vulnerability at line 36, stemming from the absence of corresponding deallocation operations for the variable across associated files. This automated risk identification enables reviewers to conduct targeted examination of the contextual code for vulnerability verification.

The test case consists of two C files (109 and 48 lines). Preprocessing requires approximately 10 ms for patch creation and 7.3 s for CPG generation, yielding a CPG with 220 nodes and 1100 edges. Analysis phase timing includes 6–8 ms for single-line patch modification detection and 146 ms for memory leak and UAF detection. The performance bottleneck lies in CPG generation during preprocessing, expected to improve with enhanced computational resources. Development teams can optimize efficiency by strategically pre-generating CPGs for critical versions during idle periods. The detection phase's minimal temporal requirements make CAPRA suitable for high-frequency update cycles in open-source development.

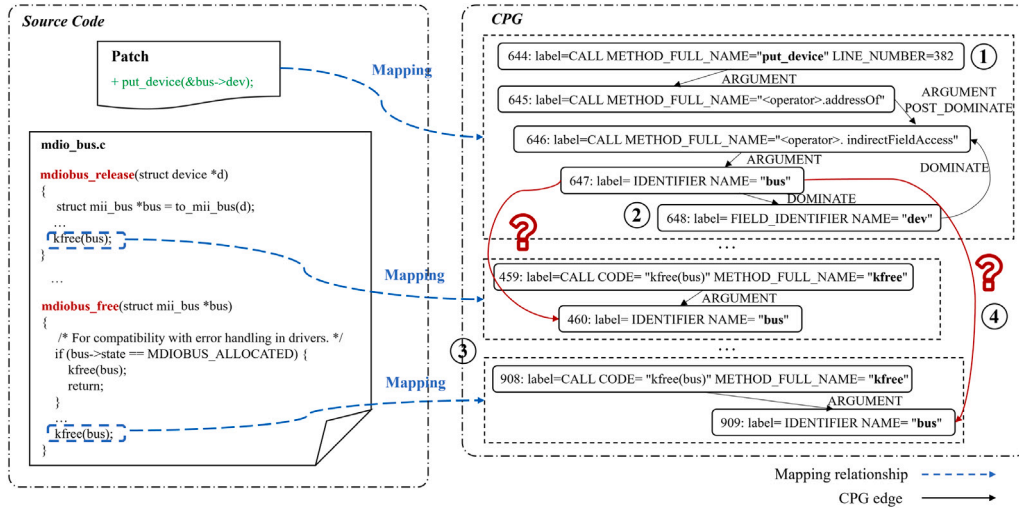


Fig. 7. UAF detection and analysis process for CVE-2019-12819. The patch adds a `put_device()` call in `mdio_bus_register()`, which CAPRA identifies as an implicit memory release operation, and tracks the usage of `bus` and its equivalent variables in the subsequent flow.

5.2.2. Case study 2

This case of an UAF vulnerability (CVE-2019-12819) originated from a patch implementing standard refcount error handling (Fig. 1). While the patch correctly prevents refcount leaks by balancing the counter on error conditions, it inadvertently triggers the `put_device()` method's implicit deallocation when the refcount reaches zero. This overlooked deallocation mechanism results in a double-free vulnerability in `mdio_bus_free()`.

CAPRA identifies memory-related risks through an extensible configuration file documenting both explicit allocation functions (`malloc()`, `kfree()`) and implicit memory operations (e.g., `put_device()`) discussed in Fig. 1). This configuration supports updates based on documented vulnerabilities and proprietary codebases. For the analyzed vulnerability, CAPRA identifies `put_device()` as a deallocation operation and traces subsequent `bus` variable usage paths, as illustrated in Fig. 7.

Step 1. Upon identifying `put_device()` insertion in the patch, CAPRA identifies this as a risky operation with implicit memory release, and locks onto the variable `bus`.

Step 2. Based on the code property graph of the source file, CAPRA records the usage paths of `bus` and its equivalent variables in the subsequent code through data flow and control flow analysis.

Steps 3-4. CAPRA analyzes `bus` usage paths by identifying various memory operations (address access, data access, array operations, deallocation, and pointer arithmetic), with operation definitions extensible via configuration files to capture implicit function call operations. For path nodes 459 and 908, the system evaluates patch integration risks by examining data reachability and control relationships between new code and memory operation nodes. UAF vulnerability risk is determined when memory related to `bus` remains accessible post-deallocation without subsequent reallocation or nullification operations in the usage path.

Resource deallocation following `mdio_bus_register()` failure exemplifies the Linux kernel's error handling protocol, where `mdio_bus_free()` deallocates the `mii_bus` structure upon driver registration failure. Analysis confined to `mdio_bus.c` proves insufficient for definitive vulnerability detection. Expanding the analytical scope to include device registration code would enable CAPRA to identify potential UAF vulnerability risks preemptively. Although CAPRA's analysis did not directly detect the UAF trigger in this commit, its risk alerts effectively guided reviewers toward targeted manual investigation.

The analyzed `mdio_bus.c` comprises 750 lines of code. Patch generation for the single modification required 7 ms. CPG generation produced a graph of 1800 nodes and 12,200 edges in 6.7 s. Despite the graph's larger size compared to previous cases, processing time

```

1  pointerA = pointerC = (char *)malloc(...);
2  ...
3  pointerB = (char *)malloc(...);
4  if (!pointerB) {
5  +   kfree(pointerA);
6     return ENOMEM;
7  }

```

Fig. 8. Code example of hypocrite commit for Case Study 3. `pointerA` and `pointerC` point to the same memory address, and the hypocrite commit in line 5 will lead to a UAF when `pointerC` continues to be used.

remained stable due to two factors: `mdio_bus.c`'s simplified logic as a device-handling module, and Joern's fixed overhead costs (JVM startup, library loading, parser initialization, and memory structure establishment).

Memory leak and UAF detection completed in 1200 ms. This phenomenon indicates that as the CPG structure expands, analysis time increases correspondingly.

5.2.3. Case study 3

This case originates from UAF case 2 in the hypocrite commit study (Wu and Lu, 2021) (Fig. 8). Prior to line 5's integration, Syzkaller+KMemleak identified a pointer leak in the error path. The researcher stealthily introduces a UAF by fixing a memory leak in the error path (line 5): `pointerA` is an alias of `pointerC`, which comes from the caller and will be further used, and thus if `pointerA` is freed improperly in an earlier location, a UAF would occur.

We enhanced the context of this hypocrite commit and successfully used CAPRA to detect the risk of triggering a UAF vulnerability in this submission, as detailed in Fig. 9. Fig. 9 highlights the code containing the immature UAF vulnerability (the red dashed box section in the source code) and associated hypocrite commit (the code that the patch attempts to add). The `allocate_and_process_memory()`'s input parameter `pointerC` shares an alias relationship with `pointerA`. Deallocation of `pointerA` creates a UAF vulnerability when executing `strcpy()` in `main()`.

As shown in Fig. 9, CAPRA's UAF detection process comprises several sequential steps:

Step 1. CAPRA identifies the memory deallocation operation and records the freed variable `pointerA` at node 72.

Step 2. CAPRA traces `pointerA`'s usage path within the current function, revealing no subsequent utilization.

Step 3. In order to examine potential external calls, CAPRA analyzes node 72's predecessor nodes, locating both the function entry point

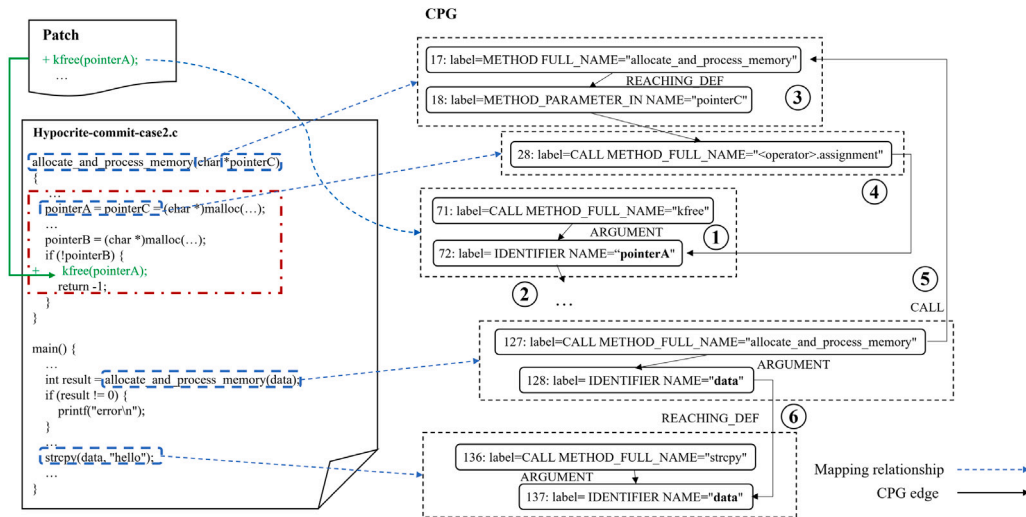


Fig. 9. UAF detection and analysis process for a hypocrite commit. The *main()* function invokes *allocate_and_process_memory()* with the local variable *data* as an argument. Within this function, memory is dynamically allocated for both the parameter and its alias *pointerA*. The hypocrite commit attack generates a UAF vulnerability by deallocating *pointerA* in an exception branch, consequently releasing the memory address referenced by the parameter.

containing *pointerA* (node 17) and the input parameter *pointerC* (node 18).

Step 4. By using static flow analysis, CAPRA determines that *pointerA* is an alias of *pointerC*. The analysis of the alias relationship for *pointerA* and its external call relationships in steps 3–4 is the key to detecting this type of hypocrite commit attack.

Step 5. CAPRA leverages control flow relationships to parse the external call site and its input parameter *data* (nodes 127, 128).

Step 6. CAPRA tracks *data*'s usage path, revealing subsequent access at node 136—indicating a UAF vulnerability.

The hypocrite commit attack essentially triggers subtle, immature vulnerabilities, thus aligning with CAPRA's detection logic. This conclusion is further validated by the detection results of other cases presented in Wu and Lu (2021).

The target file comprises 40 lines of code. CPG generation during preprocessing requires 0.5 s on average, yielding a graph with 250 nodes and 1400 edges. The moderate code volume and complexity facilitate efficient CPG generation. The subsequent analysis phase, encompassing memory leak and UAF detection, completes in 130 ms.

5.3. Additional findings

According to CAPRA's detection results, some positive memory leak cases in SARD exhibit potential immature UAF vulnerabilities. For instance, the *goodB2G2Sink()* function in sample 99220-v1.0.0 does not nullify the input parameter *data* after memory deallocation. This scenario mirrors the hypocrite commit cases analyzed in Section 5.2.2, and using such a function as a secure coding example would undoubtedly increase the risk of immature vulnerabilities. Moreover, these issues affect machine learning solutions based on this database: in multi-classification scenarios, these potential UAF vulnerabilities hidden within positive memory leak samples could interfere with the model's learning of UAF patterns.

These risks are not merely theoretical concerns. Research (Wu and Lu, 2021) has successfully demonstrated hypocrite commit attacks against the Linux kernel using analogous code patterns. Additionally, certain samples in SARD explicitly warn developers about potential vulnerabilities, as evidenced in the CWE416_UAF_char_03 test case. This test case for UAF vulnerabilities includes comments indicating potential memory leak risks.

This finding substantiates the hypothesis that developers' focus on specific vulnerabilities may inadvertently generate ancillary security

risks. The observation has significant methodological implications, particularly for machine learning research, emphasizing the necessity of critical dataset evaluation in vulnerability detection studies.

5.4. Experimental results analysis and tool limitations

5.4.1. CAPRA's detection modes

Before analyzing the experimental results, we need to explain CAPRA's detection modes. CAPRA not only provides alerts for commits that explicitly trigger known immature vulnerabilities but also identifies code submissions that may cause null pointer exceptions or introduce new immature vulnerabilities. In the statistical analysis of experimental results, we differentiated these two alerting strategies: the former was defined as strict mode, while the latter was defined as non-strict mode. *Strict mode* focuses exclusively on commits that explicitly trigger identified vulnerabilities, offering higher precision; *Non-strict mode* provides broader coverage, including potential security concerns.

This distinction is based on practical security engineering requirements. We observed that many non-targeted risk notifications (such as newly introduced immature vulnerabilities) are often viewed as false positives by developers and reviewers, as their actual harm level is relatively low, or they do not currently produce substantial impacts. Importantly, should future commits trigger these vulnerabilities, CAPRA would be able to detect them. However, from a security governance perspective, completely ignoring these risks could lead to cumulative security issues.

To balance alert precision with comprehensiveness while reducing false positive rates, CAPRA employs a dual-mode design. Strict mode ensures high-confidence alerts, suitable for security-critical scenarios requiring immediate response; Non-strict mode provides more comprehensive security monitoring, appropriate for security audits or quality assurance phases. This flexibility allows security teams to adjust CAPRA's sensitivity according to project phases and security requirements, optimizing development resource allocation without compromising security.

5.4.2. Experimental results analysis

The experimental results based on the SARD-derived dataset and the RealWorld dataset are shown in Table 1. In this context, true positives (TP) represent defects detected by CAPRA in risky commits, true negatives (TN) indicate secure commits correctly identified as risk-free, false positives (FP) denote secure commits incorrectly flagged as risky, and false negatives (FN) represent failures to detect defects

Table 1

CAPRA's detection results for commits triggering immature vulnerability patches. 'UAF' and 'MemoryLeak' datasets (SARD-based) simulate commits triggering immature UAF and memory leak vulnerabilities respectively. The RealWorld dataset includes is a mixture of legitimate commits from multiple open-source projects. All were tested in both strict and non-strict modes.

	Tuple Information	Dataset																	
Mode	Dataset Type	UAF								Memory Leak								Real -WoIrd	
		Risk				Safe				Risk				Safe				Safe	
	Patch Operation	Add	Remove	Add	Remove	Add	Remove	Add	Remove	Add	Remove	Add	Remove	Add	Remove	Add& Remove			
Metrics		TP	FN	TP	FN	TN	FP	TN	FP	TP	FN	TP	FN	TN	FP	TN	FP	TN	FP
Strict	Results	50	0	50	0	49	1	50	0	49	1	47	3	49	1	45	5	114	5
	Total	50		50		50		50		50		50		50		50		119	
Non-strict	Results	50	0	50	0	20	30	50	0	49	1	47	3	21	29	45	5	111	8
	Total	50		50		50		50		50		50		50		50		119	

in risky commits. Table 1 does not present test results related to hypocrite commits because CAPRA accurately identified all attack cases in research (Wu and Lu, 2021).

As shown in Table 1, CAPRA demonstrates excellent detection capabilities in both strict and non-strict modes across the risk sample sets for UAF and memory leaks, identifying almost all risky commits. The consistency of results between the two modes indicates not only that CAPRA performs well on this dataset, but also that it can identify target defects that truly trigger immature vulnerabilities, rather than improving tool performance by expanding the scope of risk detection. In non-strict mode, since the tool flags more potential risks beyond the target defects, it generates more 'false positives' in the secure sample set: the number of FP samples in the UAF, MemoryLeak, and RealWorld datasets increased from 1, 1, and 5 to 30, 29, and 8 respectively. The quotation marks around false positives are used because subsequent analysis of the samples revealed that the vast majority of these risks do indeed exist.

We evaluate CAPRA using five metrics: accuracy (ACC), precision, recall (TPR), false positive rate (FPR), and false negative rate (FNR), as defined in Eqs. (1)–(5):

$$ACC = \frac{(TP + TN)}{(TP + FP + TN + FN)} \quad (1)$$

$$Precision = \frac{TP}{(TP + FP)} \quad (2)$$

$$TPR = \frac{TP}{(TP + FN)} \quad (3)$$

$$FPR = \frac{FP}{(FP + TN)} \quad (4)$$

$$FNR = \frac{FN}{(FN + TP)} \quad (5)$$

Among these metrics, accuracy (ACC) represents the overall proportion of correct results (both defective and secure commits) detected by CAPRA; Precision indicates the percentage of commits flagged as defective that actually contain defects; Recall (TPR) denotes the proportion of detected defects relative to the total defective samples; False positive rate (FPR) represents the percentage of secure commits incorrectly identified as containing defects; False negative rate (FNR) indicates the proportion of defective commits misclassified as secure relative to all defective samples. The performance metrics of CAPRA on the current dataset are presented in Table 2.

In strict mode, CAPRA maintained high TPR for defect detection in both code additions and deletions that trigger immature UAF vulnerabilities, with FPR of only 2% and 0%, respectively, resulting in an overall ACC of 99.5%. CAPRA's performance declined somewhat when detecting defects that trigger potential memory leaks, particularly for defective deletion modifications, where the TPR decreased to 94%. Since CAPRA cannot calculate the reachability of conditional branches like symbolic execution, to prevent defects from evading detection through conditional statements, CAPRA defaults to considering all conditional branches as reachable and therefore does not deeply analyze

changes to conditional statements, leading to the aforementioned TPR decrease. The FPR for identifying secure commits as containing memory leak defects was approximately 10%, significantly higher than for UAF defect detection. This phenomenon is attributed to constraints in the CAPRA detection framework: when a patch simultaneously deletes both memory allocation and release code, the tool, analyzing the base file line by line before patch integration, incorrectly assumes that memory allocation behavior still exists, thereby generating false positives. However, these types of false positives are relatively easy to review, and we consider them acceptable. CAPRA ultimately maintained overall detection accuracy and precision rates of approximately 95% for memory leak defects.

In non-strict mode, due to the expanded types of risk detection, while more security risks were identified, the tool's false positive rate also increased, with comprehensive FPRs for UAF and memory leak samples rising to 30% and 34%, respectively. It should be explained that the false positive rate in non-strict mode cannot objectively indicate CAPRA's performance (only 6.7% for RealWorld dataset), because we understand CAPRA's limitations and therefore designed the legitimate commits with certain biases. Additionally, non-strict mode also generates alerts for risks of introducing immature vulnerabilities, and these risk notifications also contribute to the increased false positive rate.

CAPRA still maintained an overall ACC of around 83% in non-strict mode. Since the tool exhibited high TPR in strict mode, expanding the types of risk detection did not significantly increase the detection precision for defective samples, with results remaining largely consistent with those from strict mode. These experimental results indicate that expanding the types of risk detection not only does not actually increase false positives, but rather enables the discovery of more potential security risks introduced by patches.

Overall, within the current dataset, CAPRA's strict mode achieves good results in both defect trigger detection precision and false positive rate for legitimate commits, but loses the ability to identify a wider range of potential risks; Non-strict mode compensates for strict mode's limitations in potential risk detection, but significantly increases the false positive rate. Users can make trade-offs according to the actual project requirements.

5.4.3. Practical considerations for false positives

While CAPRA achieves relatively low false positive rates in strict mode across multiple datasets (3.5% for SARD-based dataset; 4.2% for RealWorld dataset), it is essential to address how remaining false positives, particularly in non-strict mode (32% for SARD-based dataset; 6.7% for RealWorld dataset), affect developer workload.

As shown in Fig. 2, CAPRA functions both as a standalone patch analysis tool and as a plugin within CI/CD pipelines' static code analysis phase. Unlike traditional detection tools, CAPRA analyzes only modified code segments containing potential risk operations, significantly limiting the number of generated alerts and avoiding redundant warnings for pre-existing defects in the baseline code.

Table 2

CAPRA's performance based on SARD datasets. Bold indicates metrics significantly affected in non-strict mode. Strict mode excels in ACC, Precision, and FPR but misses low-risk defects. Non-strict mode detects more defects but increases FPR, affecting ACC and Precision.

Mode	Metrics(%)	Dataset			Memory leak			Total		
		UAF			Add	Remove	All	Add	Remove	All
Strict	ACC	99	100	99.5	98	92	95	98.5	96	97.3
	Precision	98	100	99	98	90.4	94.1	98	95.2	96.6
	TPR	100	100	100	98	94	96	99	97	98
	FPR	2	0	1	2	10	6	2	5	3.5
	FNR	0	0	0	2	6	4	1	3	2
Non-Strict	ACC	70	100	85	70	92	81	70	96	83
	Precision	62.5	100	81.3	62.8	90.4	76.6	62.7	95.2	78.9
	TPR	100	100	100	98	94	96	99	97	98
	FPR	60	0	30	58	10	34	59	5	32
	FNR	0	0	0	2	6	4	1	3	2

CAPRA's line-by-line analysis approach produces alerts that are inherently more comprehensible and verifiable. When detecting vulnerabilities, CAPRA provides precise code line information and concise explanations of its reasoning. For example, if a patch adds `'free(data);'`, CAPRA identifies this as a risk operation and begins analysis. Upon finding subsequent usage of data, it generates a specific alert: *"Adding 'free(data);' may trigger a UAF vulnerability as variable 'data' is accessed later in the code."* This precision enables reviewers to quickly locate risk points and determine actual vulnerability existence, substantially reducing verification effort.

Our analysis of CAPRA's false positives reveals four primary categories:

I. Simultaneous deletion of vulnerability triggers and critical context. For instance, removing memory deallocation code for variable `data` might trigger a memory leak alert, but if the patch also removes memory allocation for `data`, a false positive occurs. Despite this architectural limitation, reviewers can quickly dismiss such alerts by examining the target file using the precise information provided.

II. Conditional logic in control flow. CAPRA cannot definitively determine code reachability in conditional branches, potentially flagging unreachable paths as security risks. While experienced developers can easily evaluate conditional logic, retaining this design helps identify hypocrite commits concealed within conditional constructs. Future work will explore LLMs-assisted approaches to mitigate this issue.

III. Extended context spans or missing files leading to absent critical nodes. For example, if developers implement a memory allocation function in a utility library without including external calls in the current patch, CAPRA cannot determine whether the memory will be manually released and thus flags a potential memory leak. However, such alerts serve a valuable purpose in highlighting external call relationships requiring review.

IV. Inaccuracy in code property graphs generated by third-party tools. Joern uses a fuzzy parsing approach (island grammars) to parse code as completely as possible, but this method may lead to incomplete parsing, especially for complex code structures (Joern, 2024). These rare occurrences will diminish as third-party tools continue to improve.

The last two categories present greater verification challenges and moderately increase manual review workload.

Comparative experiment

To further analyze developers' acceptance level of false positive rates, we selected CppCheck (Marjamäki, 2025) and TscanCode (Tencent, 2022) for a performance comparison experiment. As these tools are widely applied in real enterprise production environments, we believe that comparative experiments can indirectly reflect the impact in practical applications.

According to our research, current mainstream defect detection tools perform detection on the target files after modifications are merged. Therefore, we construct a comparative dataset based on the SARD dataset³: (i) Each sample only preserves the target file; (ii)

Table 3

Comparative experiment results. CAPRA's results are based on patch-triggered defect detection, while comparison tools detect merged files post-patch. The RealWorld dataset contains only legitimate commits and evaluates false positive performance, thus only FNR results are presented.

Dataset	Metrics (%)	CAPRA		CppCheck	TscanCode
		strict	non-strict		
SARD-derived dataset	ACC	97.3	83	56	53.3
	FNR	3.5	32	10	9.5
RealWorld dataset	ACC	–	–	–	–
	FNR	4.2	6.7	47.1	26.1

Each target file only retains code related to the patch context and vulnerability chain (excluding interference from other potential defects that might exist in the remaining code). No modifications are needed for real-world scenario dataset. The comparative experimental results are shown in Table 3.

Table 3 accuracy statistics only target UAF and memory leak vulnerabilities. Similar to CAPRA's strict and non-strict modes, tools like CheckCpp also generate different types of alerts based on risk levels. For example, CheckCpp reports include error, risk, stylistic hint, and performance type warnings, while flawfinder (Wheeler, 2023) includes 5 levels of alerts. All samples in the RealWorld dataset are code that has been vetted and reviewed by the community, so we assume these samples are secure and any alerts generated by detection tools will be considered false positives requiring additional analysis by developers.

In the SARD-derived dataset, CAPRA outperforms CppCheck's 56% and TscanCode's 53.3% in terms of accuracy metrics. Regarding false positive rates, CAPRA's strict mode performs best, but its non-strict mode at 32% is significantly higher than CppCheck's 10% and TscanCode's 9.5%.

In the RealWorld dataset, contrasting results were observed for false positive rates. CAPRA's false positive rates in its two modes were 4.2% and 6.7%, while CppCheck and TscanCode's rates increased to 47.1% and 26.1% respectively. However, these results do not necessarily indicate CAPRA's superiority over the comparison tools. Firstly, CAPRA currently only detects two types of vulnerabilities, whereas CppCheck and TscanCode support detection of multiple defect types. Secondly, CAPRA only focuses on code contained in pending patches, while the comparison tools analyze all code in target files, resulting in a broader detection scope. Finally, false positives generated by the comparison tools include numerous programming style warnings, which are easily filtered out by reviewers.

We also used other tools such as flawfinder to validate the RealWorld dataset, obtaining similar results. For example, Flawfinder

³ For detailed experimental datasets and results, please see: <https://github.com/TangBX-wtu/CAPRA/tree/main/DataSet-2025/ComparisonExperiment>.

generated 344 alerts for 33 OpenSSH samples and 237 alerts for 33 Linux samples. Although this comparative experiment is not completely rigorous due to differences in detection frameworks, it still demonstrates that CAPRA's performance is acceptable for practical applications.

In addition to our comparative experiment, there are also studies on various static detection tools. Research (Li et al., 2024) evaluated 8 mainstream static detection tools for their ability to detect smart contract vulnerabilities and found false positive rates exceeding 90%. According to experimental results from research (Goseva-Popstojanova and Perhinschi, 2015), some widely used C/C++ commercial tools have false positive rates approaching 50% when detecting vulnerabilities in CWE. These experimental results all indicate that commercial environments have a relatively high tolerance for false positive rates.

In summary, CAPRA's impact on development workflows is manageable considering: the limited number of alerts per submission, the provision of precise and easily verifiable warning information, and the predictable pattern of false positives. Apart from several categories, which moderately increase manual review effort, most false positives can be efficiently addressed without significant workflow disruption. Furthermore, this elevated false positive rate is offset by the favorable cost-benefit ratio in critical systems, where early-stage vulnerability detection substantially reduces resource expenditure compared to post-deployment remediation.

5.4.4. Limitation analysis

Due to constraints in application scenarios and technical approaches, CAPRA has certain limitations. This section provides an objective analysis from two aspects: detection capabilities and detection effectiveness.

Detection Capabilities

CAPRA's novel patch behavior analysis approach, while innovative, remains constrained by its static analysis architecture. This fundamental limitation impedes detection of resource competition-based and conditional judgment-based vulnerabilities, which often require dynamic execution context to identify reliably.

This static analysis constraint further manifests when CAPRA analyzes distributed codebases. Using a file-level analysis approach, CAPRA may lack sufficient cross-module contextual information when examining patches that span multiple modules or repositories. This can lead to incomplete analysis or false positives when dependencies extend beyond the analyzed files, particularly in large-scale systems with complex inter-module relationships.

We propose two complementary strategies to navigate these architectural constraints in large-scale implementations. Organizations might retain comprehensive scanning tools for periodic full-codebase audits while positioning CAPRA for focused protection of critical modules. Alternatively, they could augment CAPRA's configuration framework with detailed interface specifications that define cross-module memory management protocols. Such approaches reconcile the practical realities of complex systems with the imperative for comprehensive security coverage.

Detection Effectiveness

Beyond architectural limitations, detection script quality directly impacts CAPRA's performance. Currently, our implementation only supports UAF and memory leak vulnerability detection. Extending CAPRA to cover additional vulnerability types requires developers with strong security expertise, creating a potential adoption barrier in resource-constrained environments.

Additionally, the system faces a fundamental trade-off between detection sensitivity and false positive rates—a common challenge in vulnerability detection tools (Goseva-Popstojanova and Perhinschi, 2015). To mitigate late-stage remediation costs, CAPRA prioritizes comprehensive risk identification during development and review phases, particularly when defending against sophisticated threats like hypocrite

commit attacks executed through conditional constructs. This security-first approach necessarily yields higher false positive rates compared to conventional static analysis tools.

Nevertheless, based on the analysis in Section 5.4.3, the current relatively high false positive rate still imposes a manageable additional workload on developers and reviewers. We will continue to optimize CAPRA in our future work (Section 7), for example, by incorporating LLMs technology to simultaneously improve detection accuracy and enhance extensibility.

6. Related works

6.1. Bottlenecks in static analysis methods

During software development, delays in code review and static scanning hinder the detection of malicious code at the source level. Traditional static analysis tools like Sorald (Etemadi et al., 2023) and CheckCPP (Marjamäki, 2025) employ full-code scanning with time complexity proportional to code size and program complexity, failing to meet efficiency requirements in large projects with frequent commits. Research by Huang et al. (2024) developed DONAPI for NPM packages, combining static analysis with dynamic API call sequence extraction. Pan et al. (2021) proposed grammar-agnostic symbolic execution for testing complex parsing programs efficiently.

Recent advances in deep learning have enhanced software security analysis. Li et al. (2022a) introduced VulDeeLocator using code attention to identify vulnerability types. Research by Li et al. (2022b) proposed SySeVR for C/C++ vulnerability detection, while Han et al. (2024) developed bjCnet using contrastive learning for defect prediction. Large language models have also shown promise, with Dunlap et al. (2024) improving the pairing of security advisories with vulnerable functions.

Some commercial static security testing tools, such as Coverity (Black Duck Software, 2024), Fortify (OpenText, 2024), etc., have been proven to have good comprehensive detection capabilities and support multiple vulnerabilities and languages. However, full-scale scanning is not suitable for frequent execution and has higher requirements for system resources and environment configuration, the limitations of full-scale analysis persist.

6.2. Limitations of dynamic detection methods

Dynamic vulnerability detection offers superior accuracy but faces temporal challenges in large-scale development. Zhao et al. (2017) found that configuring runtime environments and executing test cases introduces significant delays, disrupting rapid iteration cycles. Research by Egele et al. (2008) confirms substantial time overhead in environment preparation, complicating integration into CI/CD pipelines and exacerbating defect detection delays.

Based on the analysis of limitations in existing detection methods, a comprehensive comparison is needed to understand how CAPRA relates to and complements these approaches. Table 4 presents a qualitative comparison between CAPRA and established vulnerability detection techniques including symbolic execution, taint analysis, fuzzing, and AI-based methods. This comparison highlights CAPRA's distinctive contributions in addressing challenges specific to high-frequency code submission environments, where traditional approaches face significant practical constraints.

As illustrated in Table 4, conventional methods such as symbolic execution, taint analysis, and fuzzing offer robust assurance in vulnerability detection. However, their computational demands and requirement to analyze the entire codebase restrict their effectiveness in fast-paced development environments. Conversely, AI-driven techniques have enhanced efficiency, though they remain largely concentrated on analyzing code after development is complete. CAPRA stands out by concentrating specifically on code patches during the review

Table 4
Qualitative comparison between CAPRA and existing vulnerability detection approaches.

Feature	Approach	CAPRA (patch-based incremental detection)	Symbolic execution	Taint analysis	Fuzzing	AI-based methods
Detection Focus		Immature vulnerabilities triggered by patches	General vulnerabilities	Data flow vulnerabilities	Input-dependent vulnerabilities	Pattern-based vulnerabilities
Integration with Workflow		Designed for code review stage	Post-development	Post-development	Post-development	Post-development
Environment Requirements		No complete environment needed	Complete environment required	Complete environment required	Executable program required	Training dataset required
Deployment Complexity		Low (Python-based)	High	Medium	High	High (requires model training)
False Positive Rate		~3.5% (strict) ~30-40% (non-strict)	Low	Medium	Low	Varies by model (~10-30%)
Detection Scope		Patch-level	Full codebase	Program execution paths	Input space exploration	Full codebase
Performance Overhead		Low (ms per line analysis)	High	Medium	High	High for training low for inference
Scalability for Large Projects		High (analyzes only modified code)	Limited	Limited	Limited	Medium (depends on model)
Hypocrite Commit Detection		Strong	Weak	Weak	None	Weak
Primary Use Case		Early detection during code review	Comprehensive security verification	Information flow tracking	Bug hunting	Large-scale security scanning
Contextual Understanding		High for code changes	High for execution paths	Medium	Low	Medium to High

process. This specialized focus allows CAPRA to efficiently identify hypocrite commit attacks while keeping deployment overhead low. Instead of replacing existing comprehensive security approaches, CAPRA functions as an initial protective layer, easing the workload on resource-intensive tools by catching potential vulnerabilities at an earlier stage in development.

6.3. Research on patch-level security analysis

Attacks carried out around malicious patches have been proven capable of bypassing most traditional static security detection tools. Research by [Gonzalez et al. \(2021\)](#) demonstrates malicious patch attacks targeting GitHub, and [Wu and Lu \(2021\)](#) revealed ‘hypocrite commit’ attacks where defects introduced through multiple patch submissions bypass static scanning.

Recent patch security research has focused on deployment status, recurring vulnerability patterns, and post-patch analysis. [Yang et al. \(2023\)](#) proposed Robin to identify recurring vulnerabilities, while [Wang et al. \(2023\)](#) developed GraphSPD using graph neural networks. [He et al. \(2023\)](#) utilized developer logs to identify vulnerable code versions, and [Xiao et al. \(2024\)](#) created FirmRec for IoT firmware.

We analyze the differences between CAPRA and existing works, as well as its additional contributions, from the following perspectives:

I. Analysis timing. Previous methods like Wang et al.’s PatchRNN ([Wang et al., 2021](#)) and Zhou et al.’s SPI ([Zhou et al., 2021](#)) excel at identifying security-relevant patches after integration, while CAPRA operates during review before patches are merged. This timing difference allows CAPRA to serve as a preventive measure during code review while other tools provide valuable post-integration security verification.

II. Patch-vulnerability relationship analysis. Existing approaches analyze relationships between patches and vulnerabilities through different lenses: [Zhang et al. \(2023\)](#) and VULDEFF ([Zhao et al., 2023](#)) focus on identifying vulnerabilities by examining patch similarities and code differences in known vulnerable functions. Sun et al.’s VDSimilar ([Sun et al., 2021](#)) advances this concept by analyzing both similarities between vulnerability pairs and differences between vulnerability-patch pairs using Siamese networks. CAPRA takes a complementary perspective by examining how pending patches might complete vulnerability trigger chains in existing code. These diverse analytical approaches demonstrate the strong connection between patches and vulnerabilities

while offering complementary insights at different stages of the security lifecycle.

III. Types of vulnerabilities of concern. PatchScope ([Zhao et al., 2020](#)), Vul-LMGNNs ([Liu et al., 2025](#)) and other traditional tools provide strong detection capabilities for mature vulnerability patterns, while CAPRA addresses immature vulnerabilities that exist in a transitional state. These complementary focuses enhance the security ecosystem’s overall effectiveness.

IV. CI/CD integration efficiency. [Wadhams et al. \(2024\)](#) focus on integrating traditional SAST tools into development pipelines, while CAPRA’s line-by-line analysis targets modified code. Each approach offers benefits depending on security requirements and development velocity.

A significant research gap exists in high-frequency iteration environments of open-source projects. While code review processes exist, research has focused on functionality ([Beller et al., 2014](#)) and automation ([Tufano et al., 2021](#)) rather than security. The CAPRA architecture addresses this critical gap in patch security analysis, complementing existing approaches by providing targeted vulnerability risk assessment during the code review phase. CAPRA offers substantial practical value for open-source projects through its ability to perform efficient, focused analysis on patch modifications—a capability not adequately addressed by existing patch security tools.

7. Discussion and future works

7.1. Challenges

Based on the analysis in Section 5.4.4, CAPRA currently faces several technical challenges. First, static analysis struggles to establish complete associations when vulnerability trigger points are distant from their occurrence locations. Second, there are no simple and efficient algorithms to accurately calculate path connectivity in conditional statements and resource competition scenarios. To reduce false negatives, CAPRA must increase detection sensitivity, which results in higher false positive rates.

For the first challenge, the industry typically relies on periodic full-scale static scanning and dynamic testing methods (such as fuzzing) to compensate for the limitations of localized analysis. For the second challenge, more complex path analysis algorithms would be necessary, but this could increase computational overhead, contradicting CAPRA’s

lightweight design principles. Additionally, the specialized knowledge required for script development limits CAPRA's extensibility to different vulnerability types.

7.2. Future research direction

To address these challenges, we plan to explore integrating large language models (LLMs) with the CAPRA architecture. CAPRA's core concept is analyzing whether pending patches complete vulnerability trigger chains. While vulnerability mechanisms are well-understood, the key challenge lies in identifying and analyzing vulnerability trigger chains. We propose creating a chain-of-thought analysis framework (similar to finite state machines) for each vulnerability type, designed based on vulnerability triggering principles. Like CAPRA's existing detection process, this chain-of-thought would determine the next analysis direction based on currently available information, while delegating code property graph static flow analysis and reachability analysis to LLMs.

Preliminary experiments suggest this approach could optimize path reachability analysis and search precision, reduce false positives, and ease script development difficulties when expanding vulnerability types. However, this direction faces several challenges, including how to fine-tune or distill knowledge into smaller parameter models (as large models would compromise CAPRA's lightweight advantage), and how to design universal chain-of-thought templates that effectively map vulnerability principles to model-recognizable rules.

Through further research and improvements, CAPRA shows promise in playing a more significant role in software supply chain security, contributing to building safer and more reliable open-source ecosystems.

8. Conclusion

This paper introduces CAPRA, a security detection methodology that analyzes pending patches to address quality risks in open-source software development. As open-source ecosystems expand, security vulnerabilities emerge from diverse coding practices and limited review processes. CAPRA employs static analysis to examine line-level patch modifications, identifying potential memory leak and UAF vulnerabilities.

CAPRA's primary contribution is its incremental approach to defect detection, focusing on patches that might activate immature vulnerabilities rather than analyzing entire codebases. This methodology detects both unintentional defects and deliberate attacks like hypocrite commit, strengthening supply chain security. The code property graph approach enables effective detection without complete compilation environments or full codebases. Additionally, its modular design accommodates project-specific requirements through independently configurable detection scripts.

Our validation using a custom dataset from SARD, historical vulnerabilities, open-source projects, and hypocrite commit research demonstrates CAPRA's effectiveness. In SARD-derived tests, CAPRA achieves 97.3% accuracy with a 3.5% false positive rate in strict mode.

Despite its effectiveness, CAPRA has limitations: detection script creation requires security expertise; static analysis struggles with dynamic execution logic; and the lenient decision approach generates higher false positives.

Future works will expand vulnerability detection types, improve algorithms to reduce false positives, explore hybrid analysis, investigate and integrate machine learning for improving performance.

In conclusion, CAPRA's distinctive design offers a new research perspective for open-source software security detection systems. Its incremental detection approach, lightweight implementation, and extensible architecture make it particularly suitable for large-scale, high-frequency iterative modern open-source projects. CAPRA effectively complements traditional solutions, working synergistically to ensure open-source software security and quality.

CRedit authorship contribution statement

Benxiao Tang: Writing – review & editing, Writing – original draft, Validation, Software, Resources, Methodology, Formal analysis, Data curation, Conceptualization. **Shilin Zhang:** Writing – review & editing, Resources, Data curation. **Fei Zhu:** Writing – review & editing, Validation, Resources. **Aoshuang Ye:** Writing – review & editing, Validation, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by Hubei Provincial Natural Science Foundation of China (Grant No. Q20231714 and No. 2025AFB323) and the Open Fund of Advanced Cryptography and System Security Key Laboratory of Sichuan Province (Grant No. SKLACSS-202313).

Data availability

I have shared my source code and data to GitHub <https://github.com/TangBX-wtu/CAPRA>, and described them on the third page of the manuscript.

References

- Alfayez, R., Behnamghader, P., Srisopha, K., Boehm, B., 2017. How does contributors involvement influence open source systems. In: 2017 IEEE 28th Annual Software Technology Conference. STC, IEEE, pp. 1–8.
- Alqudah, M., Razali, R., 2016. A review of scaling agile methods in large software development. *Int. J. Adv. Sci. Eng. Inf. Technol.* 6 (6), 828–837.
- Beller, M., Bacchelli, A., Zaidman, A., Juergens, E., 2014. Modern code reviews in open-source projects: Which problems do they fix? In: *Proceedings of the 11th Working Conference on Mining Software Repositories*. pp. 202–211.
- Black Duck Software, 2024. Coverity scan static analysis. <https://scan.coverity.com/>.
- Cadar, C., Dunbar, D., Engler, D.R., et al., 2008. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs. In: vol. OSDI, 8, pp. 209–224.
- Cai, S., Knight, K., 2013. Smatch: an evaluation metric for semantic feature structures. In: *Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. pp. 748–752.
- Coccinelle contributors, 2019. Coccinelle. <https://coccinelle.gitlabpages.inria.fr/website/>.
- Duan, X., Wu, J., Ji, S., Rui, Z., Luo, T., Yang, M., Wu, Y., 2019. VulSniper: Focus your attention to shoot fine-grained vulnerabilities. In: *IJCAI*. pp. 4665–4671.
- Dunlap, T., Meyers, J.S., Reaves, B., Enck, W., 2024. Pairing security advisories with vulnerable functions using open-source LLMs. In: *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, pp. 350–369.
- Egele, M., Scholte, T., Kirda, E., Kruegel, C., 2008. A survey on automated dynamic malware-analysis techniques and tools. *ACM Comput. Surv.* 44 (2), 1–42.
- Etemadi, K., Harrand, N., Larsén, S., Adzemovic, H., Phu, H.L., Verma, A., Madeiral, F., Wikström, D., Monperrus, M., 2023. Sorald: Automatic patch suggestions for SonarQube static analysis violations. *IEEE Trans. Dependable Secur. Comput.* 20 (4), 2794–2810. <http://dx.doi.org/10.1109/TDSC.2022.3167316>.
- Fabian, Y., Markus, L., Niko, S., Michael, P., Suchakra, S., Ursache, C.-V., 2023. Code property graph specification 1.1. <https://cpg.joern.io/#edge-ref-source-file>.
- Gonzalez, D., Zimmermann, T., Godefroid, P., Schaefer, M., 2021. Anomalous: Automated detection of anomalous and potentially malicious commits on GitHub. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). pp. 258–267. <http://dx.doi.org/10.1109/ICSE-SEIP52600.2021.00035>.
- Goseva-Popstojanova, K., Perhinschi, A., 2015. On the capability of static code analysis to detect security vulnerabilities. *Inf. Softw. Technol.* 68, 18–33. <http://dx.doi.org/10.1016/j.infsof.2015.08.002>, URL <https://www.sciencedirect.com/science/article/pii/S0950584915001366>.
- Han, J., Huang, C., Liu, J., 2024. bjCnet: A contrastive learning-based framework for software defect prediction. *Comput. Secur.* 145, 104024. <http://dx.doi.org/10.1016/j.cose.2024.104024>, URL <https://www.sciencedirect.com/science/article/pii/S0167404824003298>.

- He, Y., Wang, Y., Zhu, S., Wang, W., Zhang, Y., Li, Q., Yu, A., 2023. Automatically identifying cve affected versions with patches and developer logs. *IEEE Trans. Dependable Secur. Comput.* 21 (2), 905–919.
- Huang, C., Wang, N., Wang, Z., Sun, S., Li, L., Chen, J., Zhao, Q., Han, J., Yang, Z., Shi, L., 2024. DONAPI: Malicious NPM packages detector using behavior sequence knowledge mapping. In: 33rd USENIX Security Symposium (USENIX Security 24). USENIX Association, Philadelphia, PA, pp. 3765–3782, URL <https://www.usenix.org/conference/usenixsecurity24/presentation/huang-cheng>.
- Joern, 2024. Joern documentation. <https://docs.joern.io/>.
- Klees, G., Ruef, A., Cooper, B., Wei, S., Hicks, M., 2018. Evaluating fuzz testing. In: *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. pp. 2123–2138.
- Larrosa, A., 2024. Fix a small memory leak. <https://github.com/openssh/openssh-portable/pull/515>.
- Le, T.H.M., Croft, R., Hin, D., Babar, M.A., 2021. A large-scale study of security vulnerability support on developer q&a websites. In: *Proceedings of the 25th International Conference on Evaluation and Assessment in Software Engineering. EASE '21, Association for Computing Machinery, New York, NY, USA*, pp. 109–118. <http://dx.doi.org/10.1145/3463274.3463331>.
- Li, F., Paxson, V., 2017. A large-scale empirical study of security patches. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. CCS '17, Association for Computing Machinery, New York, NY, USA*, pp. 2201–2215. <http://dx.doi.org/10.1145/3133956.3134072>.
- Li, K., Xue, Y., Chen, S., Liu, H., Sun, K., Hu, M., Wang, H., Liu, Y., Chen, Y., 2024. Static application security testing (SAST) tools for smart contracts: How far are we? *Proc. ACM Softw. Eng.* 1 (FSE), <http://dx.doi.org/10.1145/3660772>.
- Li, Z., Zou, D., Xu, S., Chen, Z., Zhu, Y., Jin, H., 2022a. VulDeeLocator: A deep learning-based fine-grained vulnerability detector. *IEEE Trans. Dependable Secur. Comput.* 19 (4), 2821–2837. <http://dx.doi.org/10.1109/TDSC.2021.3076142>.
- Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z., 2021. Sysevr: A framework for using deep learning to detect software vulnerabilities. *IEEE Trans. Dependable Secur. Comput.* 19 (4), 2244–2258.
- Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z., 2022b. SySeVR: A framework for using deep learning to detect software vulnerabilities. *IEEE Trans. Dependable Secur. Comput.* 19 (4), 2244–2258. <http://dx.doi.org/10.1109/TDSC.2021.3051525>.
- Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., Zhong, Y., 2018. Vuldeecker: A deep learning-based system for vulnerability detection. *arXiv preprint arXiv:1801.01681*.
- Lin, G., Zhang, J., Luo, W., Pan, L., De Vel, O., Montague, P., Xiang, Y., 2019. Software vulnerability discovery via learning multi-domain knowledge bases. *IEEE Trans. Dependable Secur. Comput.* 18 (5), 2469–2485.
- Liu, R., Wang, Y., Xu, H., Sun, J., Zhang, F., Li, P., Guo, Z., 2025. Vul-LMGNNs: Fusing language models and online-distilled graph neural networks for code vulnerability detection. *Inf. Fusion* 115, 102748.
- Marjamäki, D., 2025. Cppcheck: static analysis of C/C++ code. <https://github.com/danmar/cppcheck/>.
- Meneely, A., Williams, L., 2009. Secure open source collaboration: an empirical study of linus' law. In: *Proceedings of the 16th ACM Conference on Computer and Communications Security. CCS '09, Association for Computing Machinery, New York, NY, USA*, pp. 453–462. <http://dx.doi.org/10.1145/1653662.1653717>.
- Nakayama, M., Hustad, E., Sutcliffe, N., 2021. Agility and system documentation in large-scale enterprise system projects: a knowledge management perspective. *Procedia Comput. Sci.* 181, 386–393.
- Newsome, J., Song, D.X., 2005. Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software. In: *NDSS*, vol. 5, Citeseer, pp. 3–4.
- Nguyen, T.G., Le-Cong, T., Kang, H.J., Le, X.-B.D., Lo, D., 2022. Vulcrator: a vulnerability-fixing commit detector. In: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. pp. 1726–1730.
- NVD, 2023. CVE-2023-4004 detail. <https://nvd.nist.gov/vuln/detail/CVE-2023-4004>.
- OpenText, 2024. OpenText fortify static code analyzer. <https://www.opentext.com/products/fortify-static-code-analyzer>.
- Pan, W., Chen, Z., Zhang, G., Luo, Y., Zhang, Y., Wang, J., 2021. Grammar-agnostic symbolic execution by token symbolization. In: *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis. In: ISSTA 2021, Association for Computing Machinery, New York, NY, USA*, pp. 374–387. <http://dx.doi.org/10.1145/3460319.3464845>.
- Rebouças, M., Santos, R.O., Pinto, G., Castor, F., 2017. How does contributors' involvement influence the build status of an open-source software project? In: *2017 IEEE/ACM 14th International Conference on Mining Software Repositories. MSR, IEEE*, pp. 475–478.
- Sun, H., Cui, L., Li, L., Ding, Z., Hao, Z., Cui, J., Liu, P., 2021. Vdsimilar: Vulnerability detection based on code similarity of vulnerabilities and patches. *Comput. Secur.* 110, 102417.
- Synopsys, 2024. 2024 open source security and risk analysis report. <https://www.synopsys.com/zh-cn/software-integrity/resources/analyst-reports/open-source-security-risk-analysis.html>.
- Tencent, 2022. TscanCode: A static code analyzer for C++, C#, Lua. <https://github.com/Tencent/TscanCode>.
- The kernel development community, 2024. Everything you ever wanted to know about linux -stable releases. <https://www.kernel.org/doc/html/v6.9/process/stable-kernel-rules.html>.
- Tufano, R., Pascarella, L., Tufano, M., Poshvanyk, D., Bavota, G., 2021. Towards automating code review activities. In: *2021 IEEE/ACM 43rd International Conference on Software Engineering. ICSE, IEEE*, pp. 163–174.
- Wadhams, Z., Reinhold, A.M., Izurieta, C., 2024. Automating static code analysis through CI/CD pipeline integration. In: *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering-Companion (SANER-C)*. IEEE, pp. 119–125.
- Wang, X., Wang, S., Feng, P., Sun, K., Jajodia, S., Benchaaboun, S., Geck, F., 2021. Patchrnn: A deep learning-based system for security patch identification. In: *MILCOM 2021-2021 IEEE Military Communications Conference. MILCOM, IEEE*, pp. 595–600.
- Wang, S., Wang, X., Sun, K., Jajodia, S., Wang, H., Li, Q., 2023. Graphspd: Graph-based security patch detection with enriched code semantics. In: *2023 IEEE Symposium on Security and Privacy. SP, IEEE*, pp. 2409–2426.
- Wheeler, D.A., 2023. Flawfinder: a static analysis tool for finding vulnerabilities in c/c++ source code. <https://github.com/david-a-wheeler/flawfinder>.
- Wickman, B., Hu, H., Yun, I., Jang, D., Lim, J., Kashyap, S., Kim, T., 2021. Preventing Use-After-Free attacks with fast forward allocation. In: *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, pp. 2453–2470, URL <https://www.usenix.org/conference/usenixsecurity21/presentation/wickman>.
- Wu, Q., He, Y., McCamant, S., Lu, K., 2020. Precisely characterizing security impact in a flood of patches via symbolic rule comparison. In: *In Network and Distributed Systems Security Symposium, NDSS*. <http://dx.doi.org/10.14722/ndss.2020.24419>.
- Wu, Q., Lu, K., 2021. On the feasibility of stealthily introducing vulnerabilities in open-source software via hypocrite commits. *Proc. OaKl.* 17.
- Xiao, H., Zhang, Y., Shen, M., Lin, C., Zhang, C., Liu, S., Yang, M., 2024. Accurate and efficient recurring vulnerability detection for IoT firmware. In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. pp. 3317–3331.
- Yamaguchi, F., Golde, N., Arp, D., Rieck, K., 2014. Modeling and discovering vulnerabilities with code property graphs. In: *2014 IEEE Symposium on Security and Privacy. IEEE*, pp. 590–604.
- Yang, S., Xu, Z., Xiao, Y., Lang, Z., Tang, W., Liu, Y., Shi, Z., Li, H., Sun, L., 2023. Towards practical binary code similarity detection: Vulnerability verification via patch semantic analysis. *ACM Trans. Softw. Eng. Methodol.* 32 (6), 1–29.
- Zajdel, S., Costa, D.E., Mili, H., 2022. Open source software: an approach to controlling usage and risk in application ecosystems. In: *Proceedings of the 26th ACM International Systems and Software Product Line Conference - Volume a. SPLC '22, Association for Computing Machinery, New York, NY, USA*, pp. 154–163. <http://dx.doi.org/10.1145/3546932.3547000>.
- Zhang, Z., Wang, Y., Zhao, Y., Sun, X., 2023. Patch-based vulnerability discovery techniques. In: *2023 IEEE International Conference on Sensors, Electronics and Computer Engineering. ICSECE, IEEE*, pp. 1371–1382.
- Zhao, Q., Huang, C., Dai, L., 2023. VULDEFF: Vulnerability detection method based on function fingerprints and code differences. *Knowl.-Based Syst.* 260, 110139.
- Zhao, Y., Serebrenik, A., Zhou, Y., Filkov, V., Vasilescu, B., 2017. The impact of continuous integration on other software development practices: a large-scale empirical study. In: *2017 32nd IEEE/ACM International Conference on Automated Software Engineering. ASE, IEEE*, pp. 60–71.
- Zhao, L., Zhu, Y., Ming, J., Zhang, Y., Zhang, H., Yin, H., 2020. PatchScope: Memory object centric patch diffing. In: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. pp. 149–165.
- Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y., 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. *Adv. Neural Inf. Process. Syst.* 32.
- Zhou, Y., Siow, J.K., Wang, C., Liu, S., Liu, Y., 2021. Spi: Automated identification of security patches via commits. *ACM Trans. Softw. Eng. Methodol.* (TOSEM) 31 (1), 1–27.